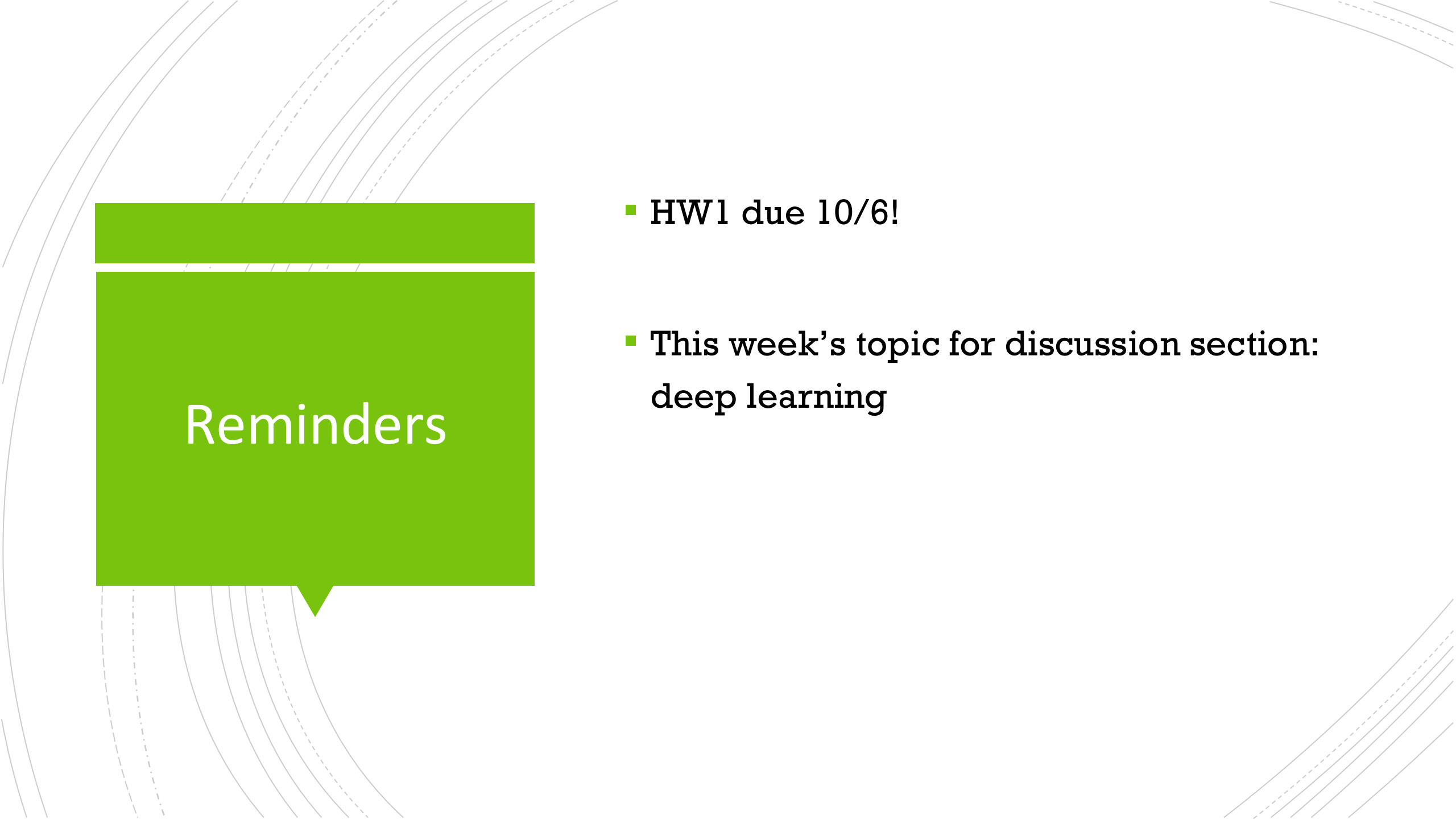


Deep Learning 1

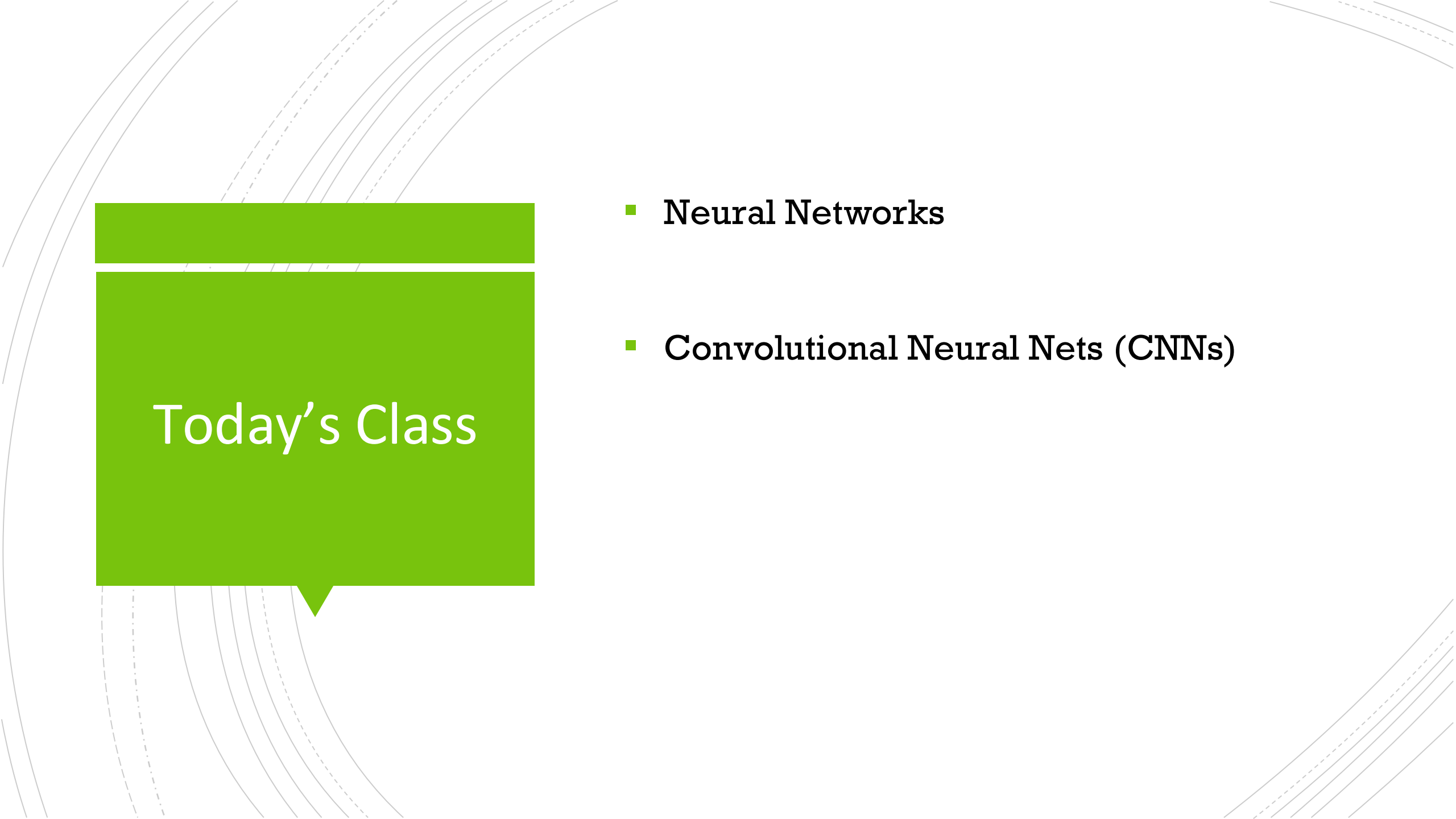
EECS 492

Introduction to Artificial Intelligence

The background of the slide features several thin, curved lines in a light gray color, some solid and some dashed, creating a subtle, abstract pattern.

Reminders

- HW1 due 10/6!
- This week's topic for discussion section:
deep learning

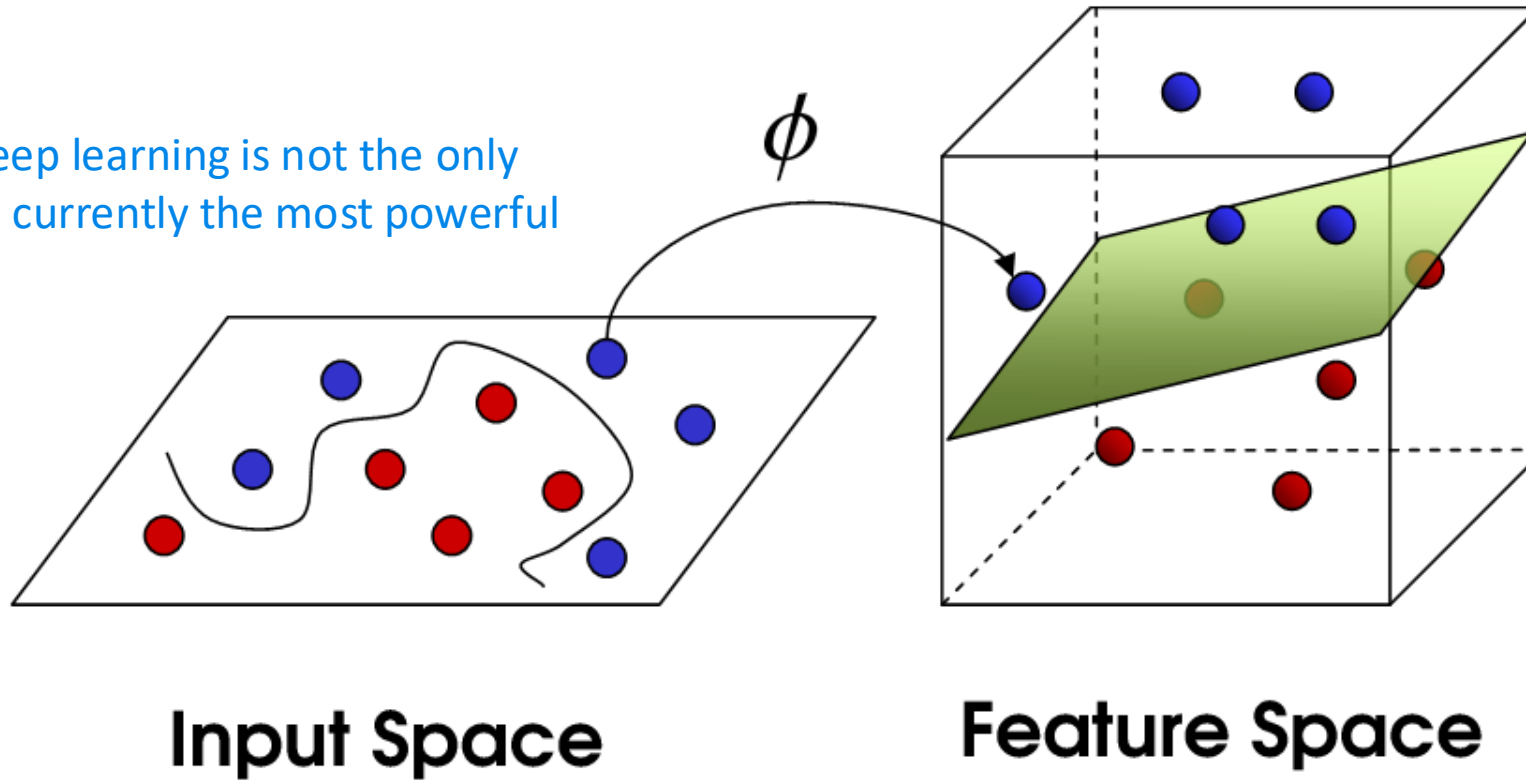
The background of the slide features a series of thin, curved lines in shades of gray, creating a sense of motion and depth. These lines are more prominent on the left side and fade towards the right.

Today's Class

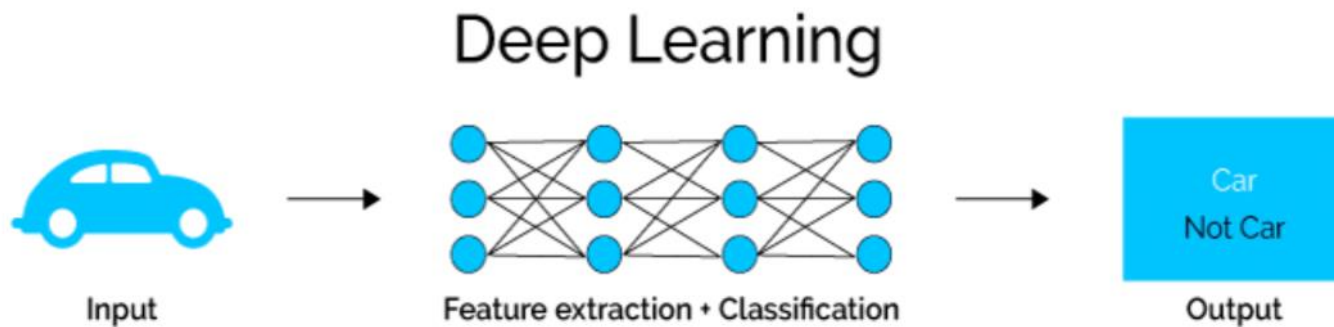
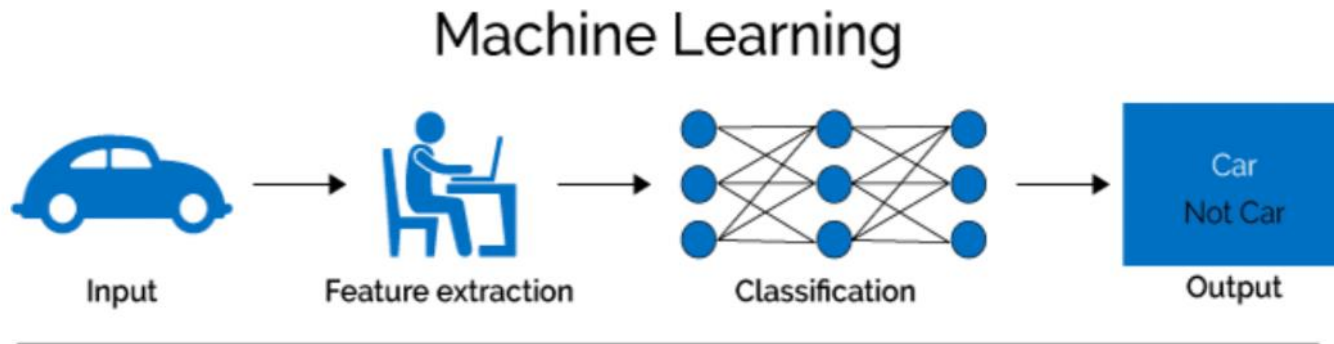
- **Neural Networks**
- **Convolutional Neural Nets (CNNs)**

Neural Networks Enable Automated Feature Engineering

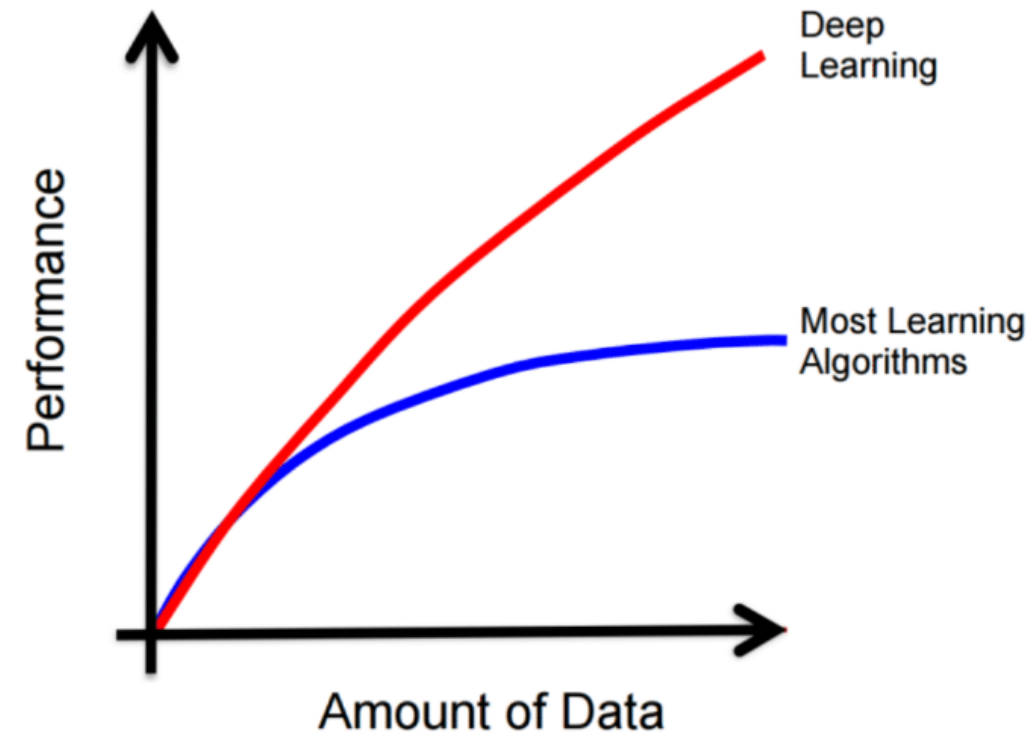
Note: Deep learning is not the only one, but currently the most powerful



Deep Learning Automates the Creation of Knowledge Representations



<https://deeplearning.mit.edu/>



Neural Network Unit

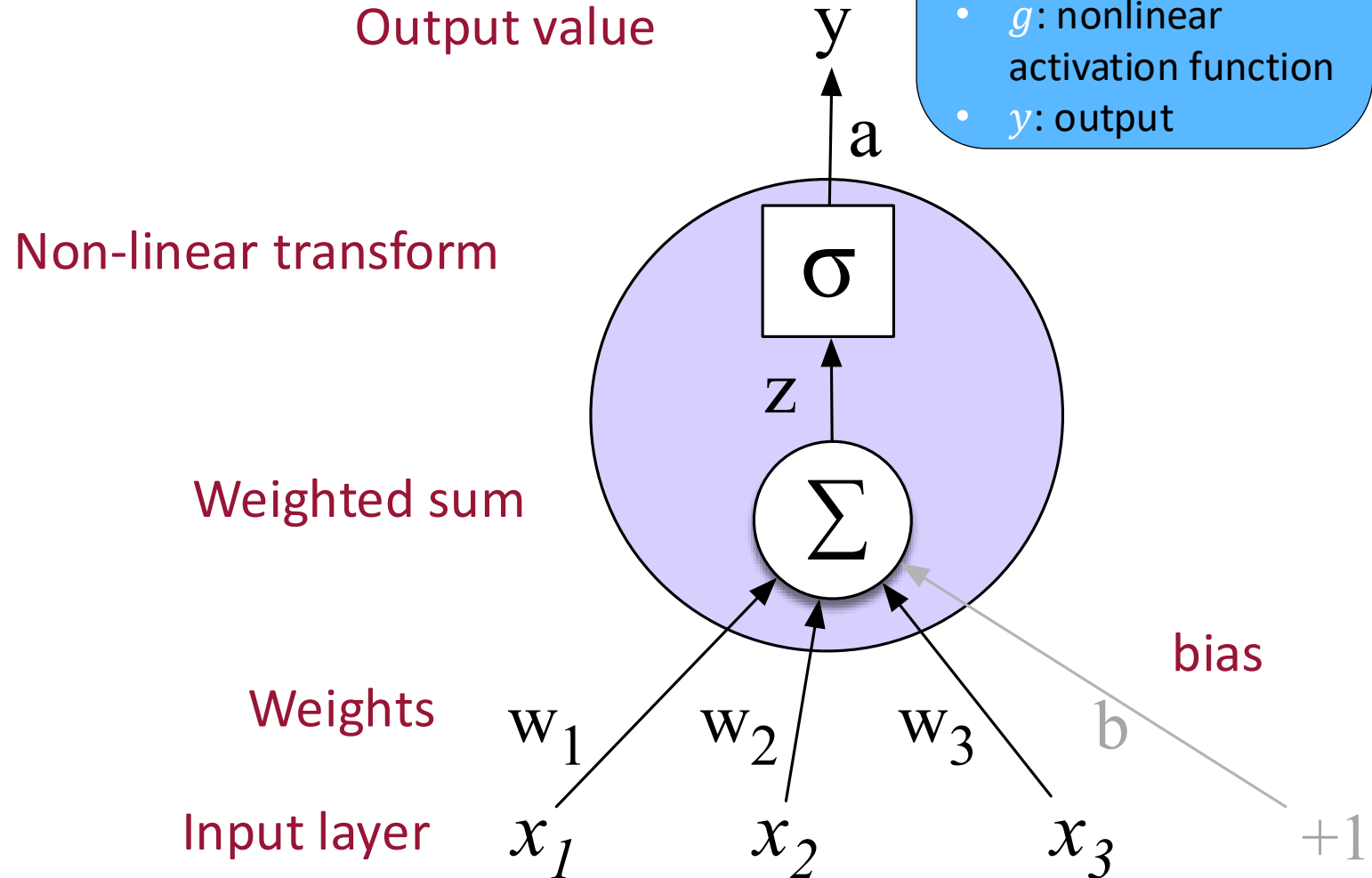
- Take weighted sum of inputs, plus a bias

$$z = b + \sum_i w_i x_i$$

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

- Instead of just using z , we'll apply a nonlinear activation function g :

$$y = a = g(z)$$

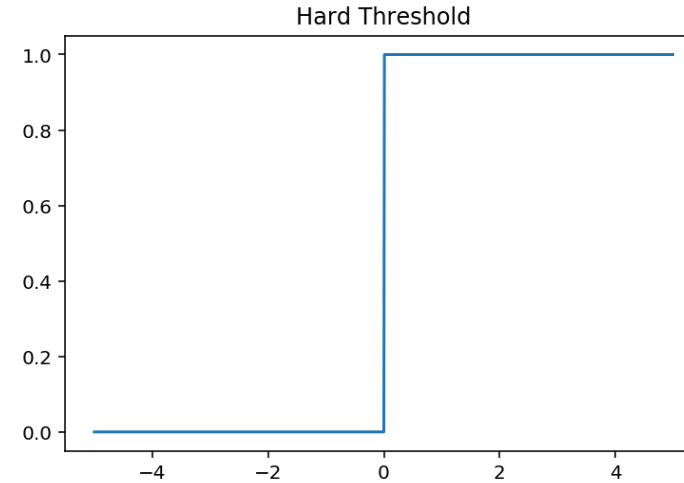


Types of Activation Functions

- Hard threshold

$$g(s) = 1 \text{ iff } s > 0, \text{ else } 0$$

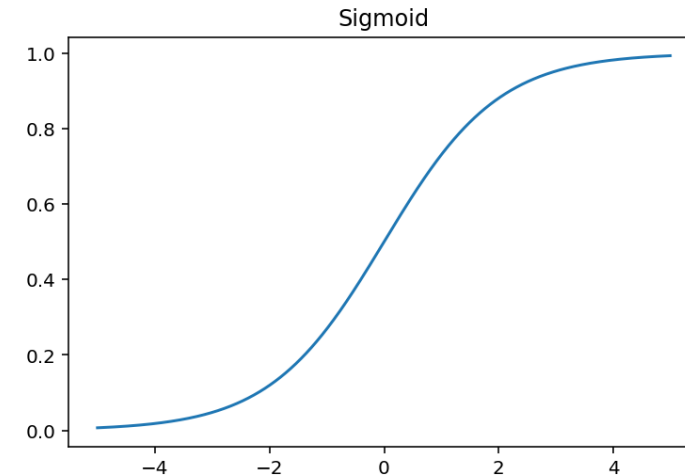
Non-differentiable!



- Sigmoid threshold

$$g(s) = \frac{1}{1 + e^{-s}}$$

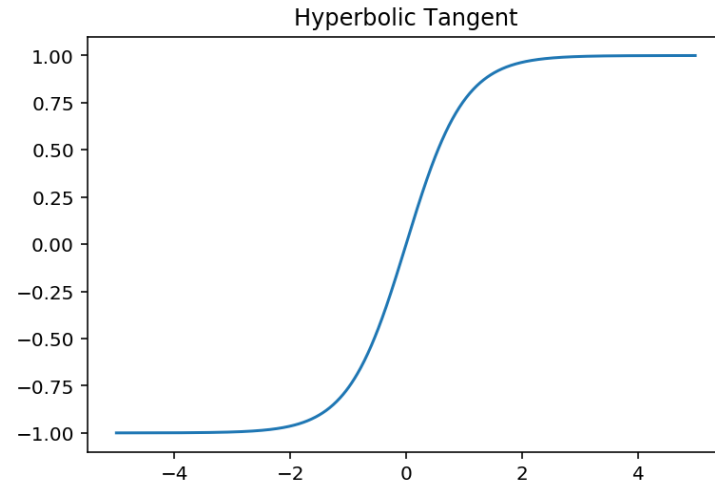
$$g'(s) = g(s)(1 - g(s))$$



Types of Activation Functions

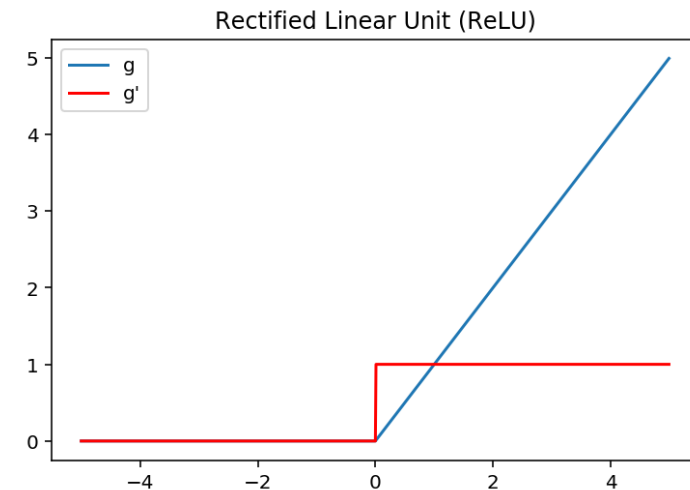
- Hyperbolic Tangent (tanh)

$$g(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$



- Rectified Linear Unit (ReLU)

$$g(s) = \max(0, s)$$
$$g'(s) = \begin{cases} 1, & s > 0 \\ 0, & \text{otherwise} \end{cases}$$



This is just the hard threshold.
Leads to sparse activations!



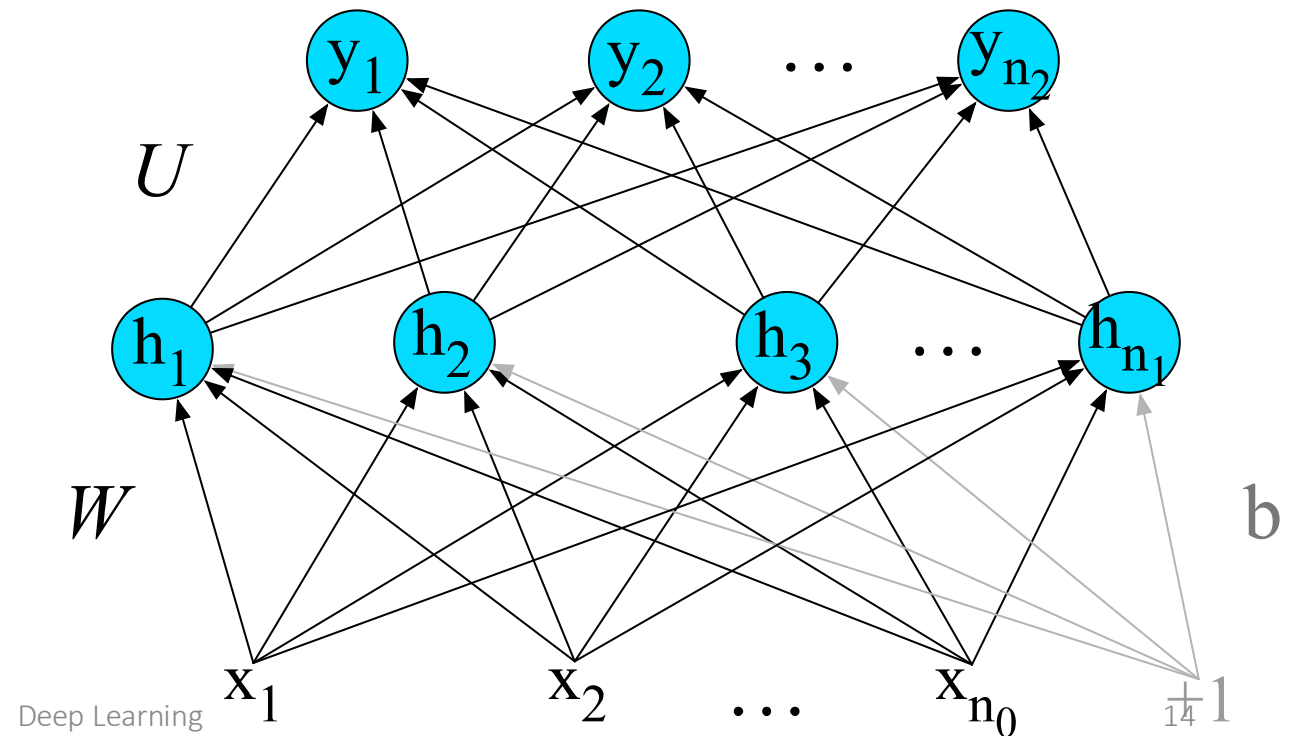
Example

- Suppose a unit has:
 - $\mathbf{w} = [0.2, 0.3, 0.9]$
 - $b = 0.5$
 - Sigmoid activation function $\rightarrow \sigma(s) = \frac{1}{1 + e^{-s}}$
- What is the output y for input vector $\mathbf{x} = [0.5, 0.6, 0.1]$?

- x_i : i th input
- W, U : weight matrices

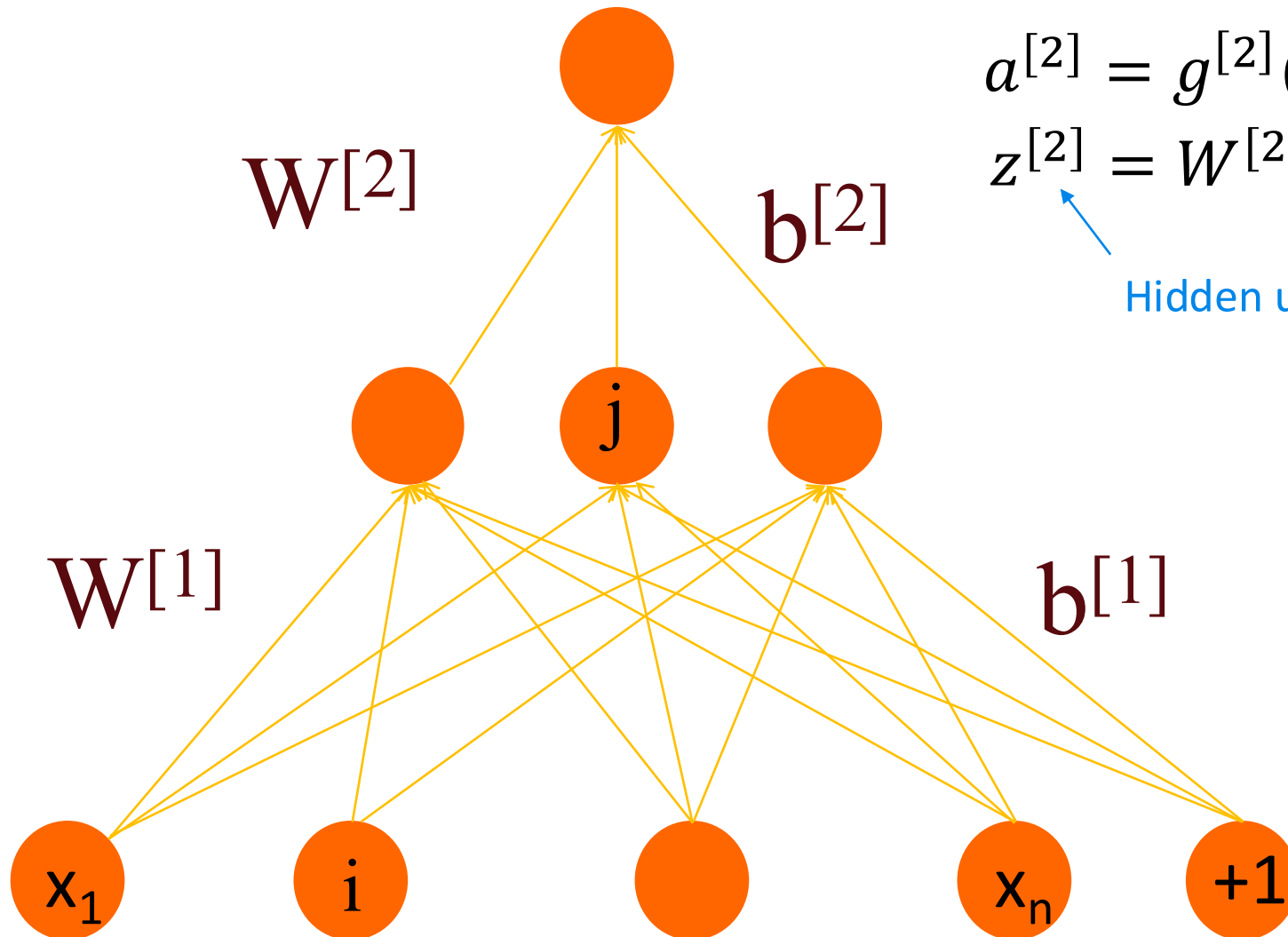
Feedforward Neural Networks (FNN)

- Multilayer network in which the units are connected with no cycles
 - Outputs from units in each layer are passed to units in the next higher layer
 - No outputs are passed back to lower layers
- Each layer is **fully-connected**
 - Link between every pair of units from 2 adjacent layers
- Also known as:
 - Multi-layer perceptron (**MLP**)
 - Fully-connected Neural Net (**FCNN**)



Multi-layer Notation

- x_i : i th input
- W, U : weight matrices
- a : activation value
- b : bias term
- z : output before nonlinear function
- g : nonlinear activation function



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

Hidden unit, often denoted with letter z or h

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[0]}$$

Multi Layer Notation

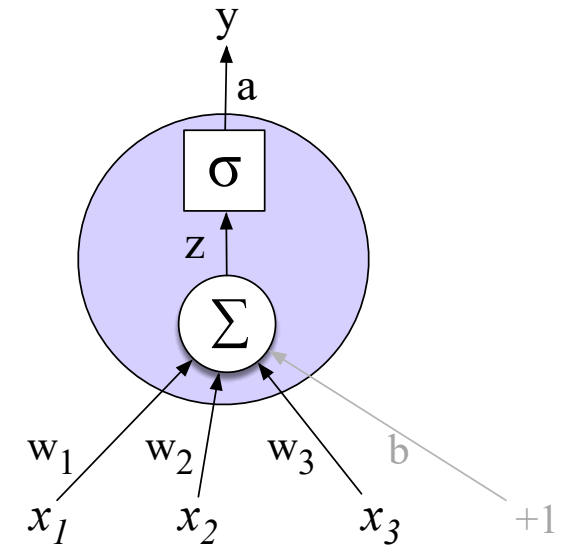
$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$\hat{y} = a^{[2]}$$



for i in $1..n$

$$z^{[i]} = W^{[i]}a^{[i-1]} + b^{[i]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

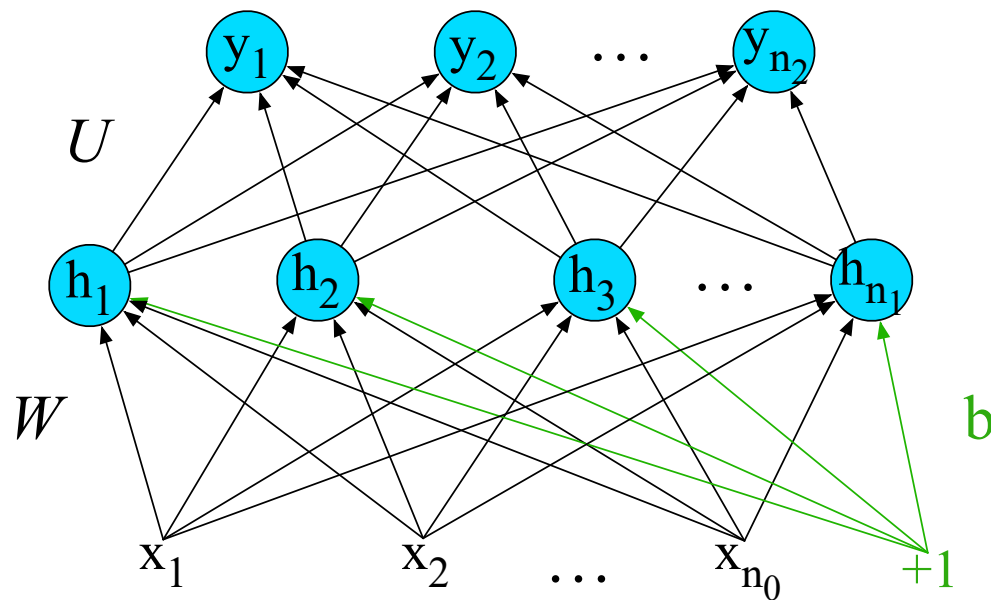
$$\hat{y} = a^{[n]}$$

Replacing the bias unit

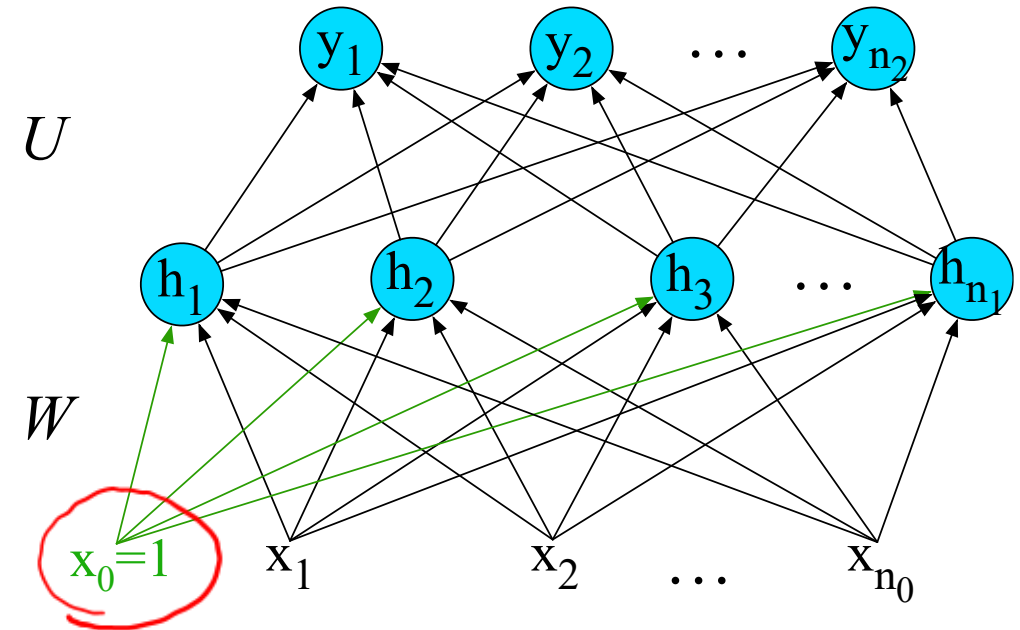
- Let's switch to a notation without the bias unit
- Just a notational change
 1. Add node $a_0=1$ to each layer
 2. Its weight w_0 will be the bias
 3. So input layer $a^{[0]}_0=1$,
 - And $a^{[1]}_0=1$, $a^{[2]}_0=1, \dots$

Replacing the bias unit

Instead of:



We'll do this:



Replacing the bias unit

- Instead of:

$$x = x_1, x_2, \dots, x_{n_0}$$

$$h = g \left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j \right)$$

$$h = g(Wx + b)$$

We'll do this:

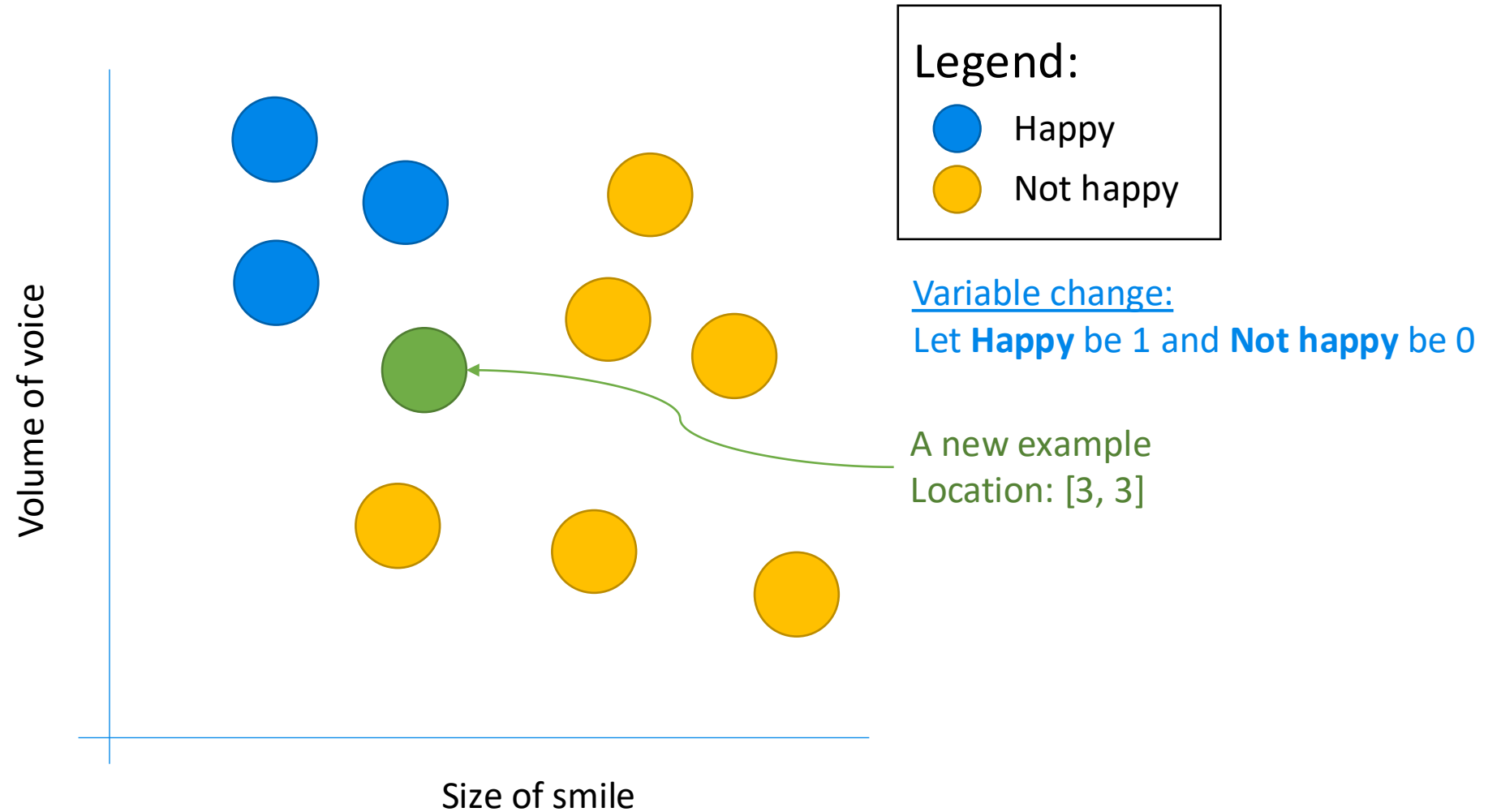
$$x = \boxed{x_0}, x_1, x_2, \dots, x_{n_0}$$

$$h = g \left(\sum_{i=0}^{n_0} W_{ji} x_i \right)$$

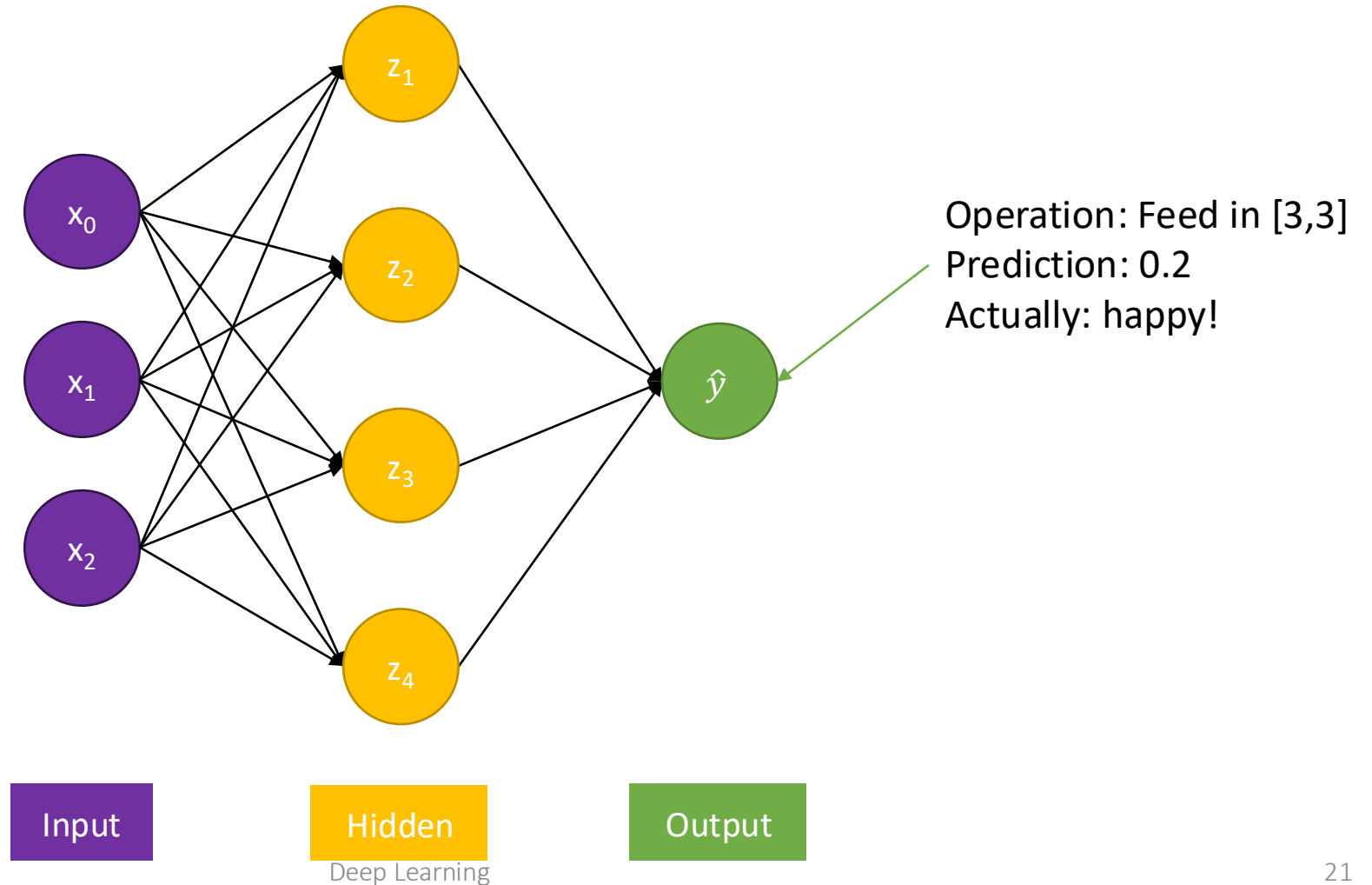
$$h = g(Wx)$$

Easier to handle!
(do this only for the
input layer)

A toy example: Am I happy?

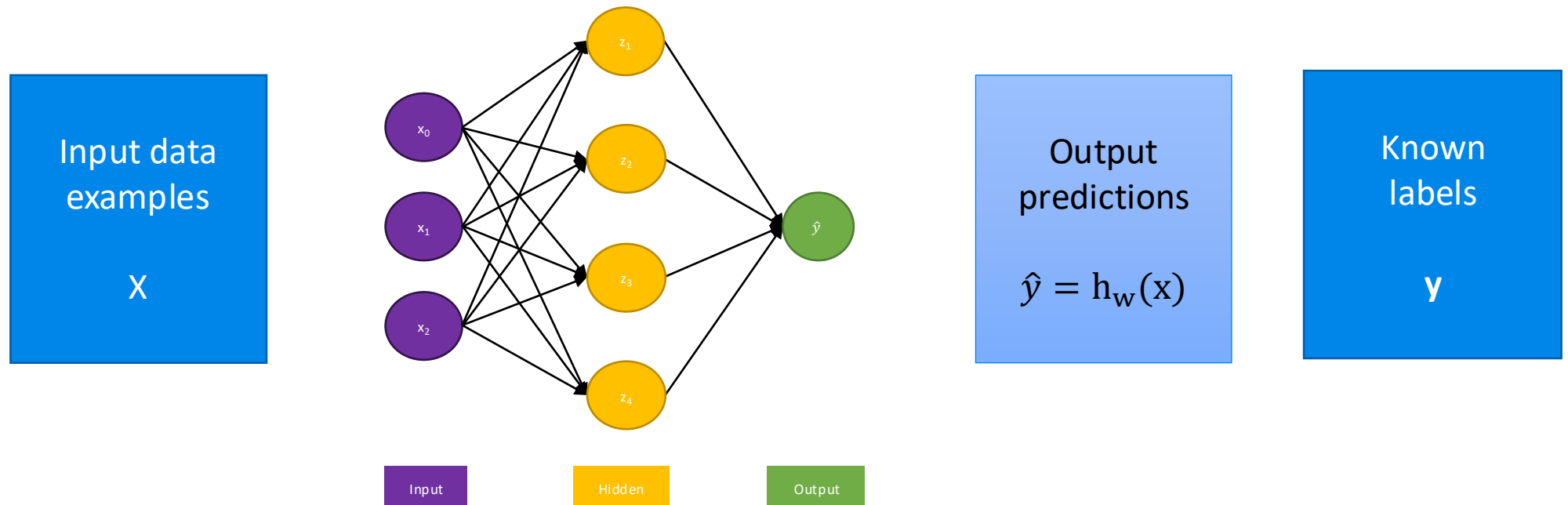


A toy example: Am I happy?



How close was I?

- Common approach: **binary cross entropy loss**
 - Used when known labels are binary (only 2 classes)



Cross Entropy

- Remember entropy?
 - Entropy = measure of uncertainty (higher entropy, more uncertainty)
 - $H(V) = -\sum_k P(v_k) \log_2 P(v_k)$
- Related idea: **cross entropy**
 - It measures how well one distribution B approximates a true distribution A

$$H(A, B) = - \sum_k \underbrace{P(a_k)} \log_2 \underbrace{P(b_k)}$$

k : possible outcomes (classes) of the random variable

- In supervised learning:
 - **A** represents the **true** (data-generating) distribution
 - **B** is the model's **predicted** distribution

Binary Cross Entropy Loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels

- We can use cross entropy as a loss
 - y – the true labels (labeled 0 or 1)
 - $h_W(x)$ - the labels predicted by the neural network

$$Loss(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(h_W(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_W(x^{(i)}))$$

Cross entropy for
the true labels with
class 1

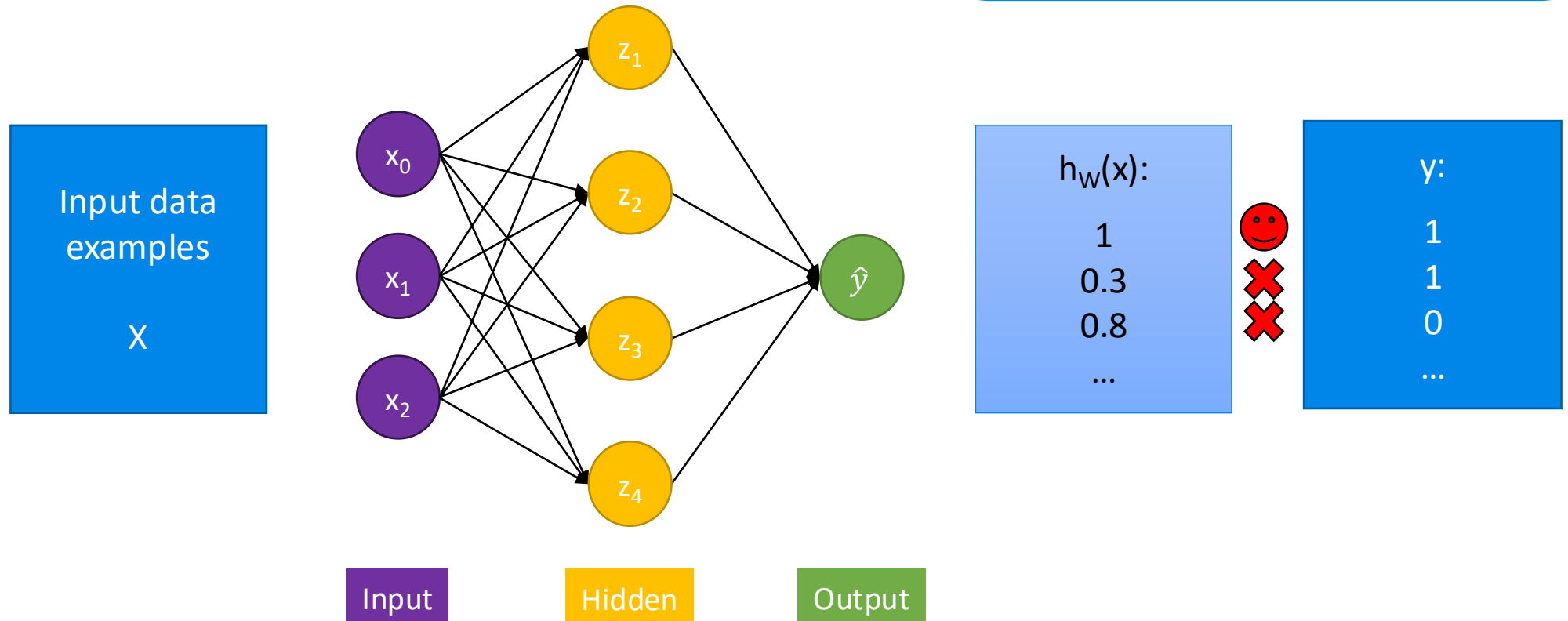
Cross entropy for
the true labels with
class 0

How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels

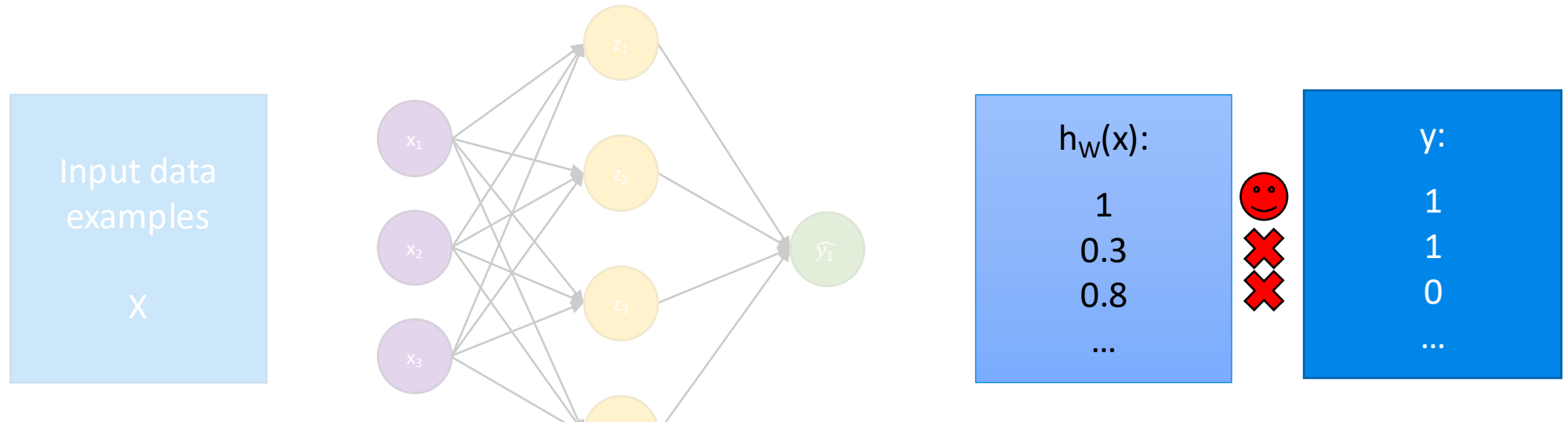


How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Formulate loss:

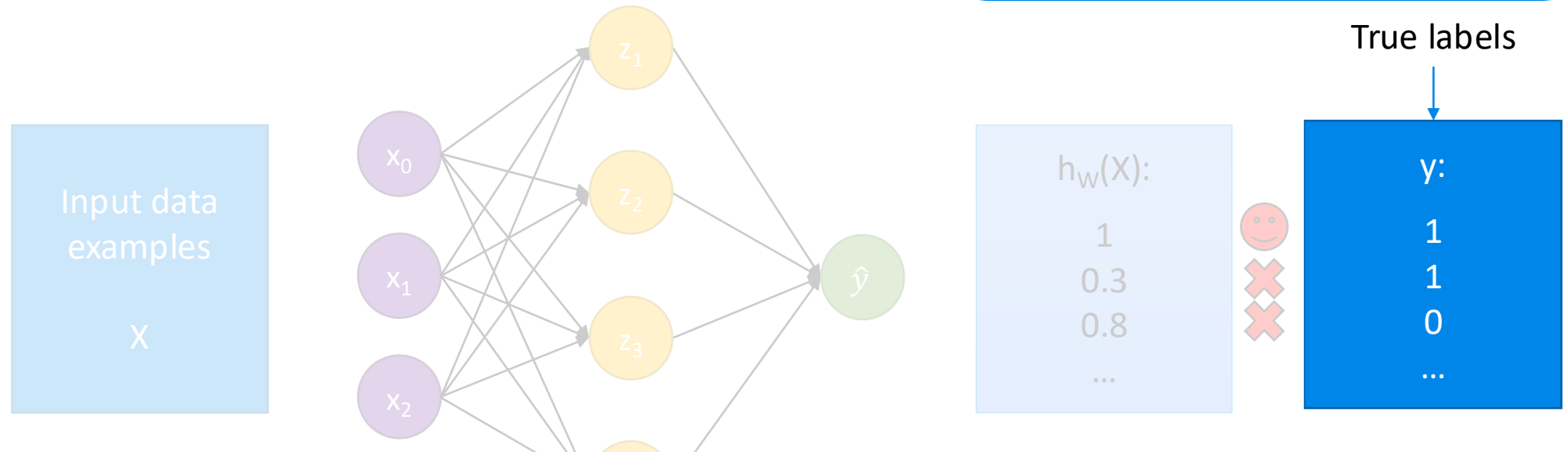
$$Loss(W) = -\frac{1}{n} \sum_i y^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)}))$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Formulate loss:

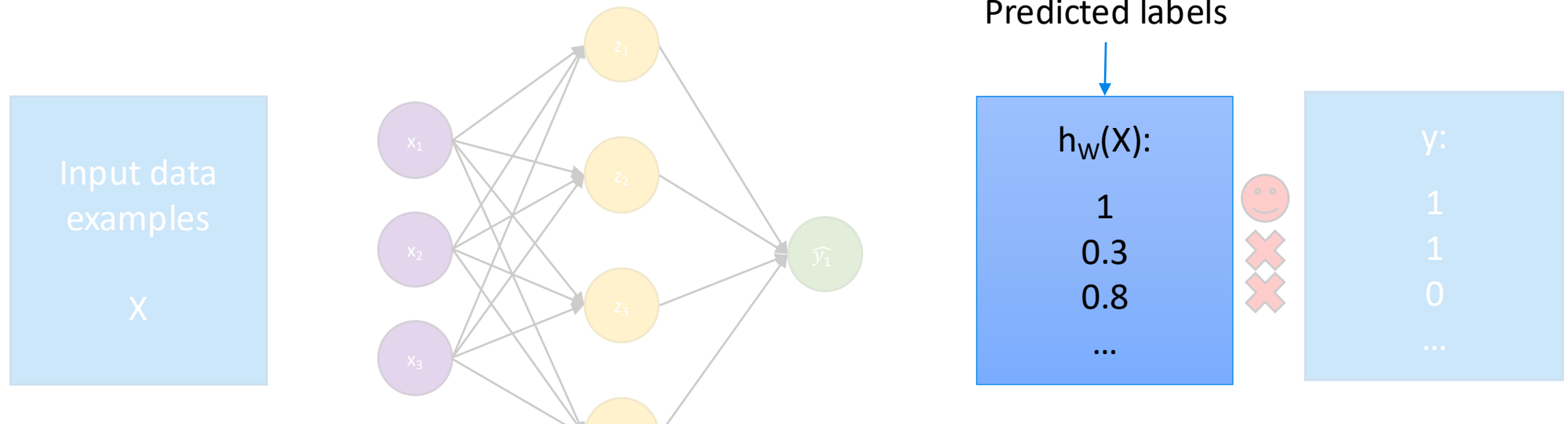
$$Loss(W) = -\frac{1}{n} \sum_i \mathbf{y}^{(i)} \log(h_w(x^{(i)})) + (1 - \mathbf{y}^{(i)}) \log(1 - h_w(x^{(i)}))$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Formulate loss:

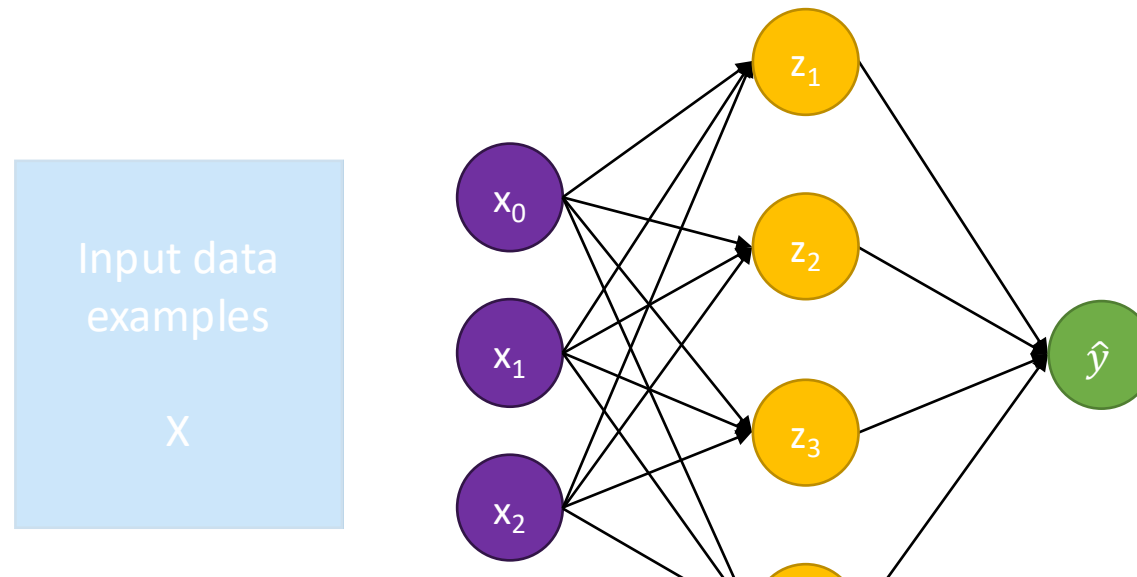
$$Loss(W) = -\frac{1}{n} \sum_i y^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)}))$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Items that are actually happy

$h_w(X)$:		y :
1	😊	1
0.3	❌	1
0.8	❌	0
...		...

Formulate loss:

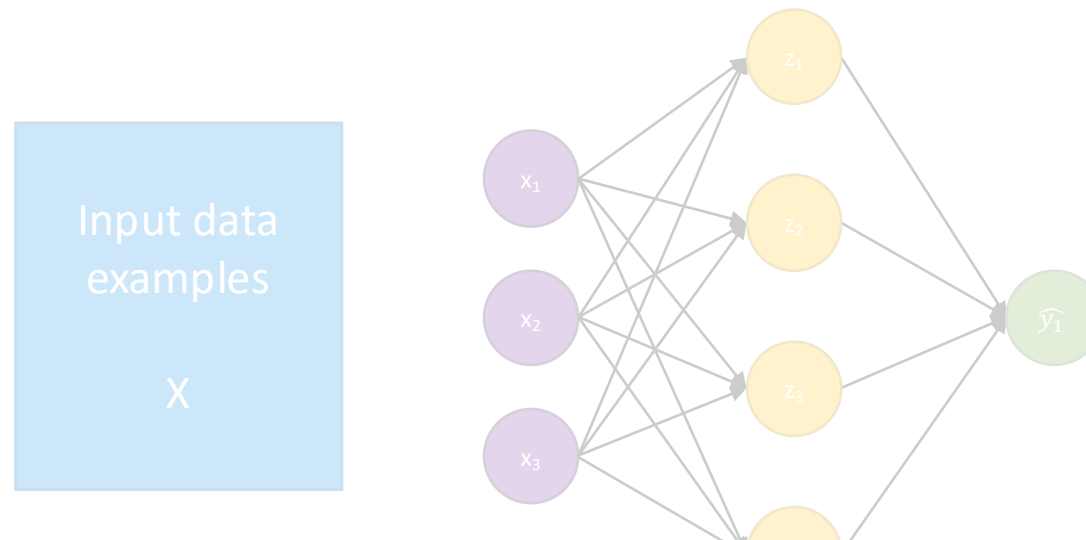
$$Loss(W) = -\frac{1}{n} \sum_i \mathbf{y}^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)}))$$

How close was I?

- Common approach: binary cross entropy loss

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Items that are actually not happy

$h_w(X)$:		y :
1	😊	1
0.3	❌	1
0.2	❌	0
...		...

Formulate loss:

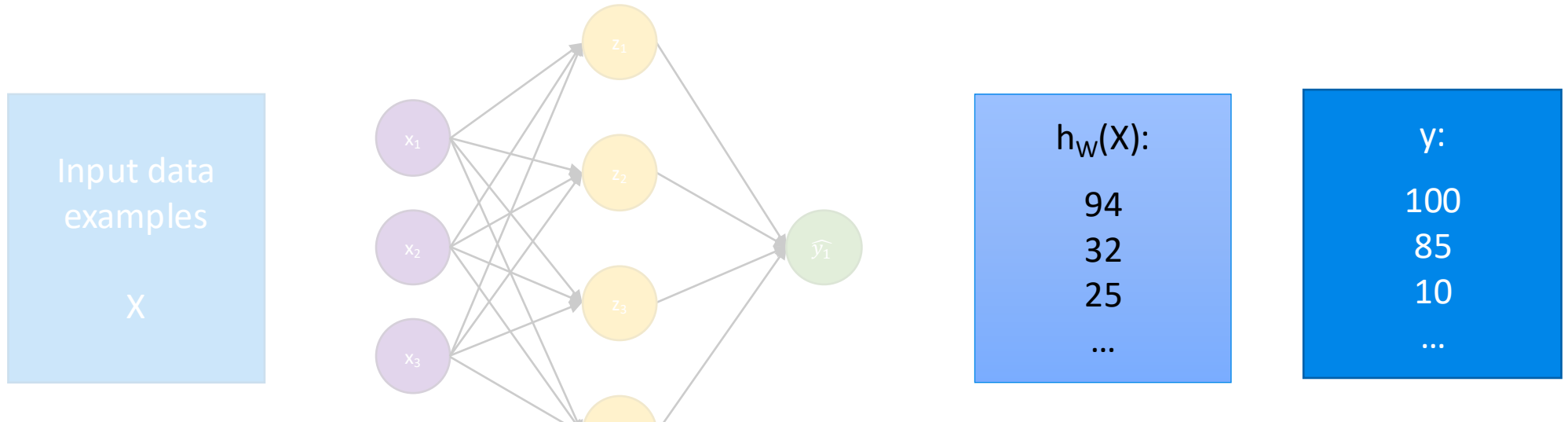
$$Loss(W) = -\frac{1}{n} \sum_i y^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)}))$$

How close was I?

- Common approach: mean squared error (MSE) loss (how happy am I?)

Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(X)$: the estimated labels
- y : the “true” labels



Formulate loss:

$$Loss(W) = \frac{1}{n} \sum_i \left(y^{(i)} - h_w(x^{(i)}) \right)^2$$

Now we can train the model!

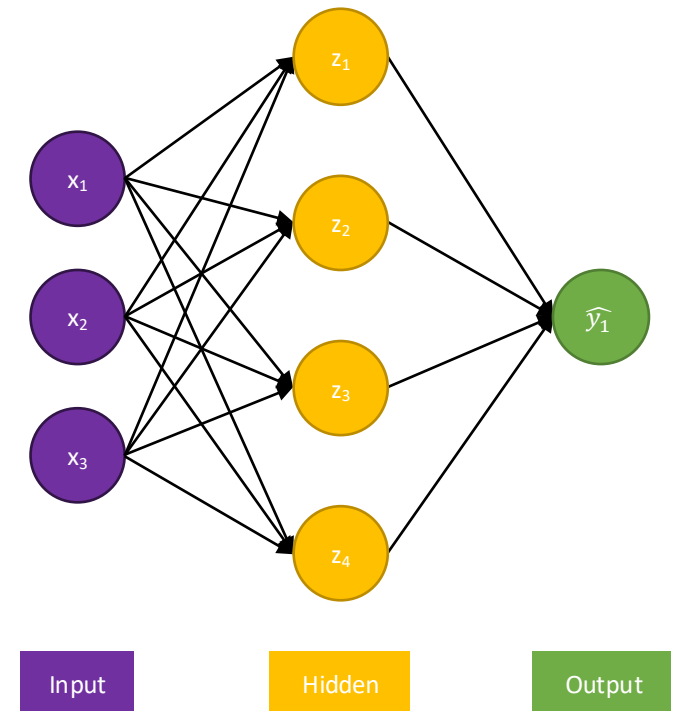
- We would like to *minimize* loss

$$W^* = \underset{W}{\operatorname{argmin}} \underbrace{\frac{1}{n} \sum_i \left(\mathcal{L}(h_W(x), y^{(i)}) \right)}_{\text{Loss}(W)}$$

$$W^* = \underset{W}{\operatorname{argmin}} (\text{Loss}(W))$$

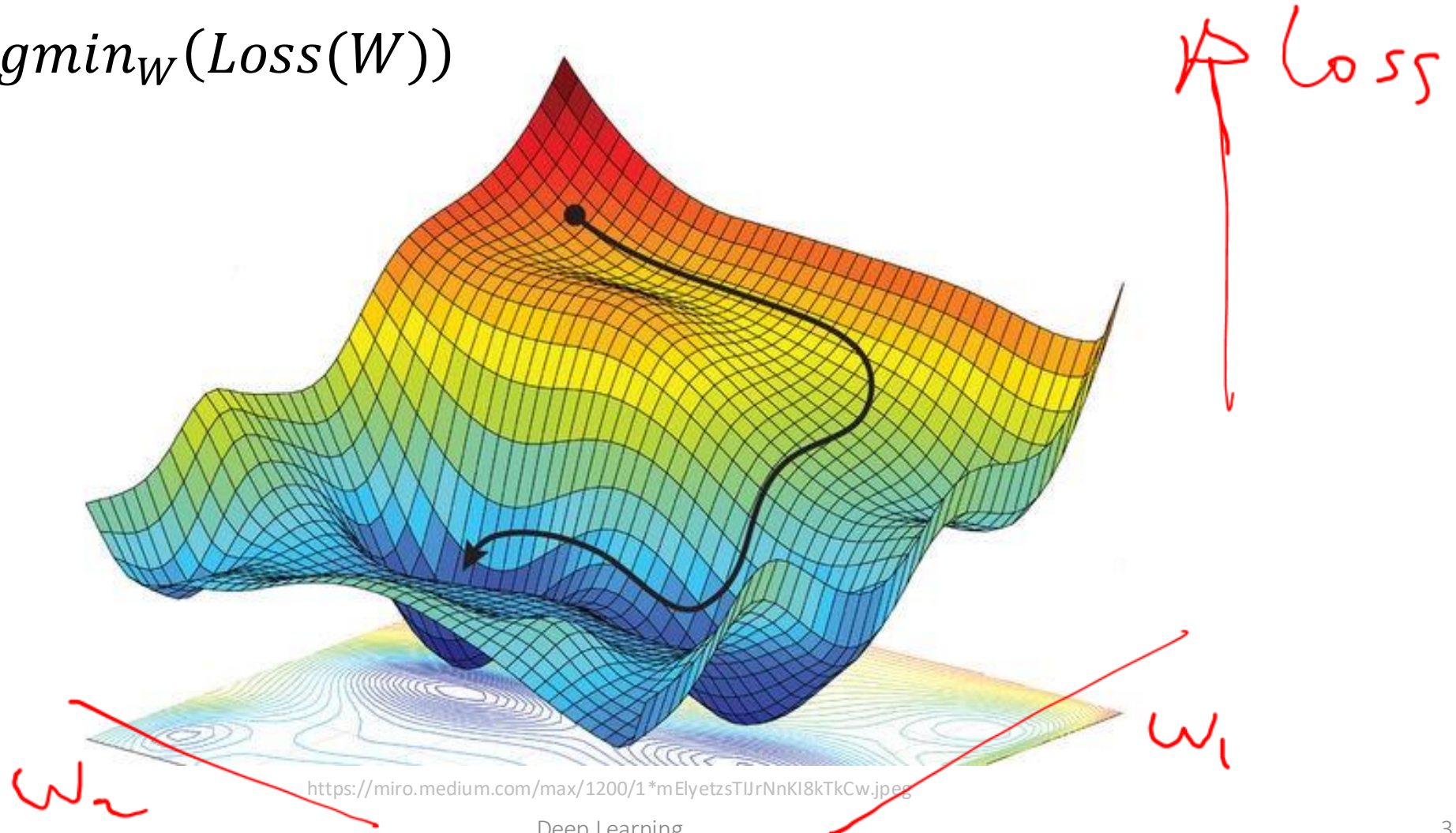
Notation:

- X : the input matrix
- z_i : the hidden units
- s_i : the output layer, pre-nonlinearity
- W : the weight matrix
- $\hat{y}/h_w(x)$: the estimated labels
- y : the “true” labels



We need to find the right weights

$$W^* = \operatorname{argmin}_W (\operatorname{Loss}(W))$$



Optimization Methods for Neural Networks

- Batch Gradient descent
- Stochastic gradient descent

Batch Gradient Descent

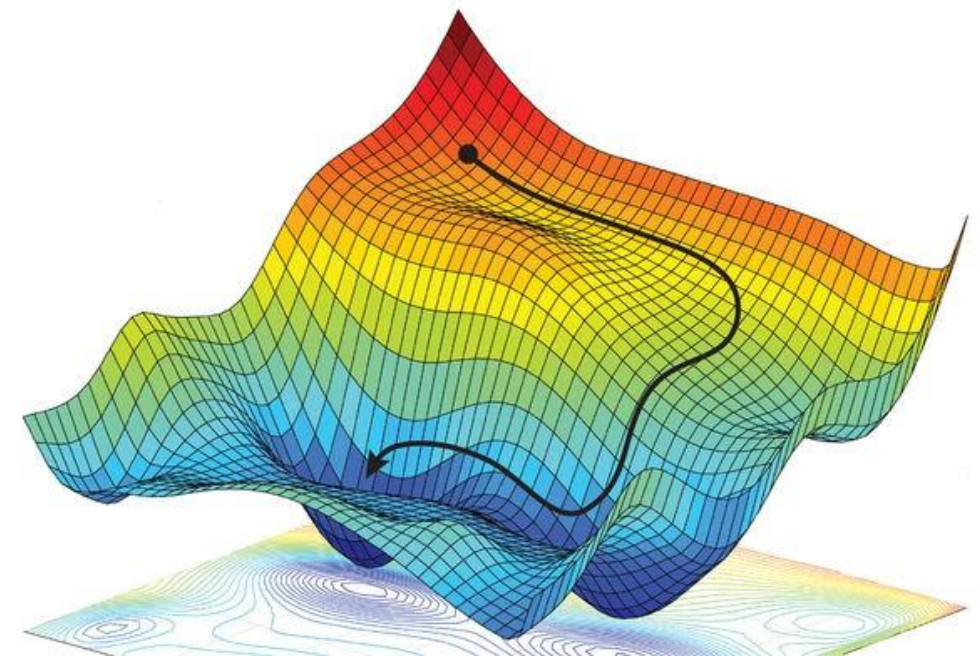
Notation:

- W : the weight matrix
- α : the learning rate

- Initialize weights randomly
- Loop until convergence:
 - Compute gradient: $\frac{\partial Loss(W)}{\partial W}$
 - Update weights: $W \leftarrow W - \alpha \frac{\partial Loss(W)}{\partial W}$
- Return weights

Learning rate

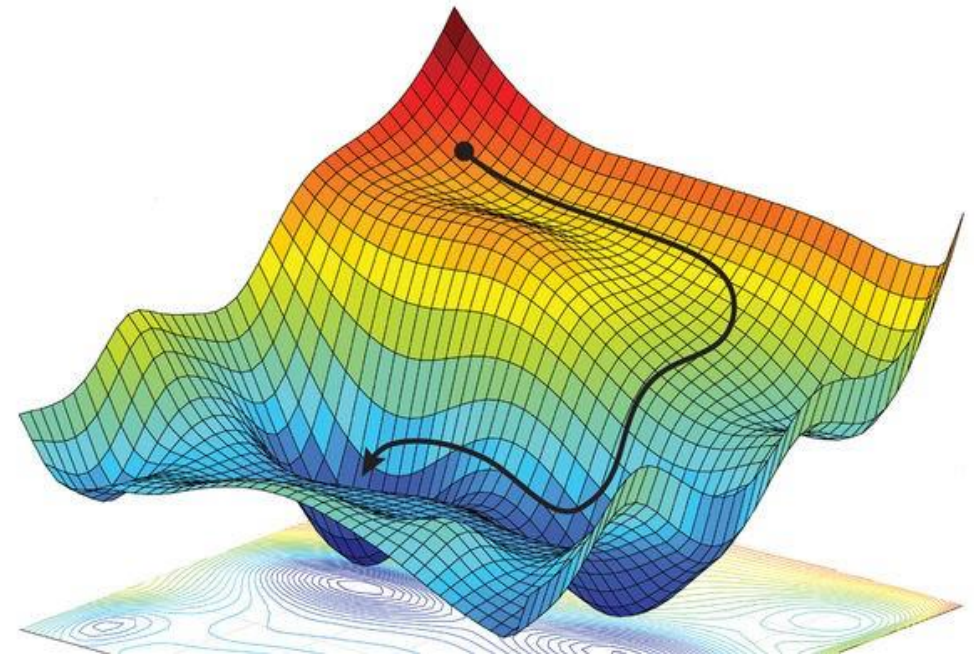
Move "downhill"



https://miro.medium.com/max/1200/1*mElyetzsTlJrNnKI8kTkCw.jpeg



What happens if the learning rate is too small? Too big?



https://miro.medium.com/max/1200/1*mElyetzsTUrNnKI8kTkCw.jpeg

Changing the learning rate

*Note: it is not necessary that the learning rate be fixed!

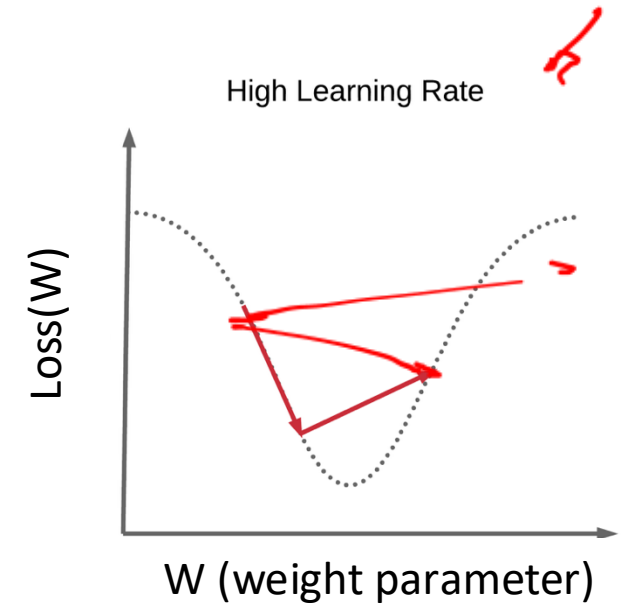
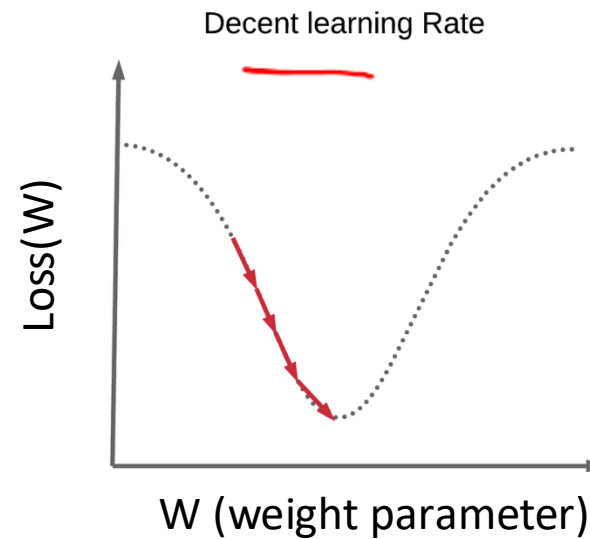
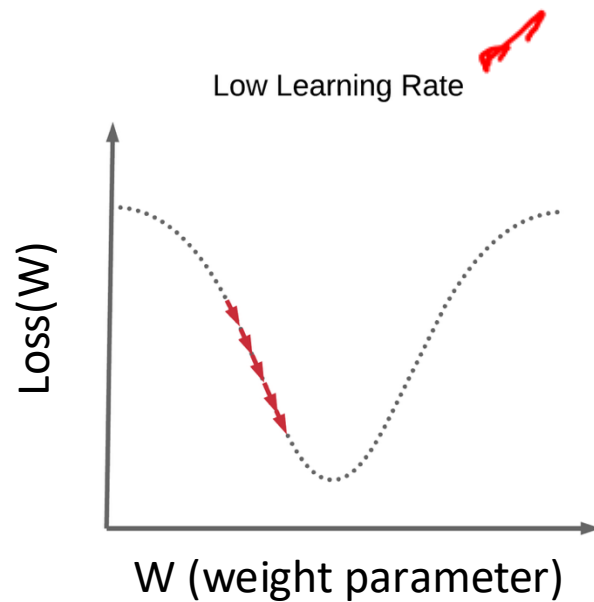


Image source: https://www.deeplearningwizard.com/deep_learning/boosting_models_pytorch/lr_scheduling/

Stochastic Gradient Descent (SGD)

- Batch gradient descent is expensive when N is large!
- Instead, SGD would update for every single example
- More efficient: approximate sum using a **minibatch** of examples
 - 32 / 64 / 128 common
 - Update weights with minibatch gradient

Minibatch Gradient Descent

Initialize weights randomly

Loop until convergence:

Shuffle dataset

Loop over minibatch:

Compute gradient for minibatch $\frac{\partial \text{Loss}(W)}{\partial W}$

Update weights: $W \leftarrow W - \alpha \frac{\partial \text{Loss}(W)}{\partial W}$

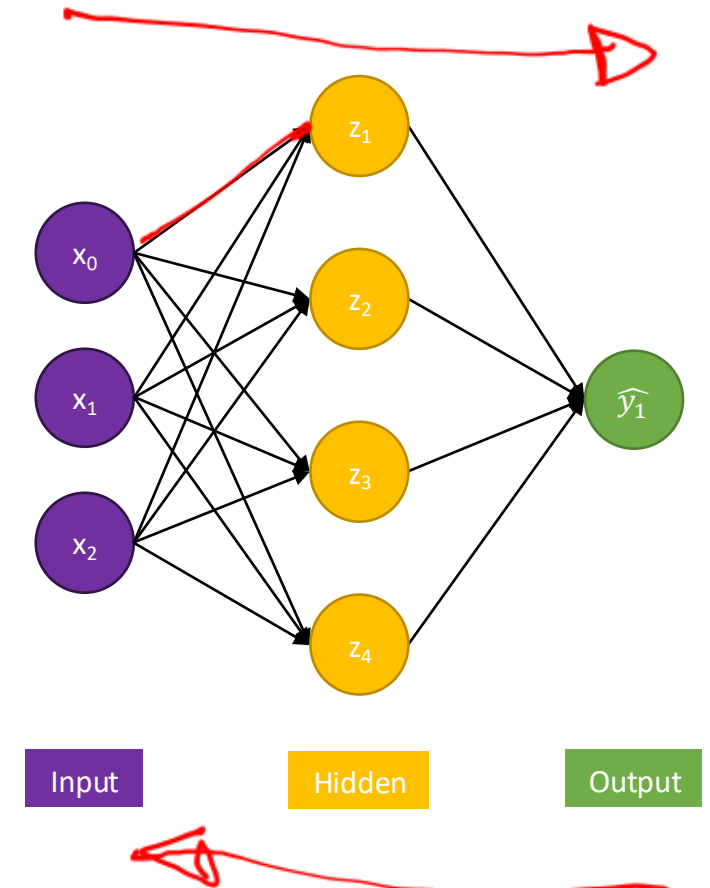
Return weights

Backpropagation

Backpropagation

$$\frac{\partial L}{\partial w}$$

- Basic idea:
 - Compute effect of every weight on output
 - Work backwards from output to input
- If output is the wrong value – move weights in direction of gradient



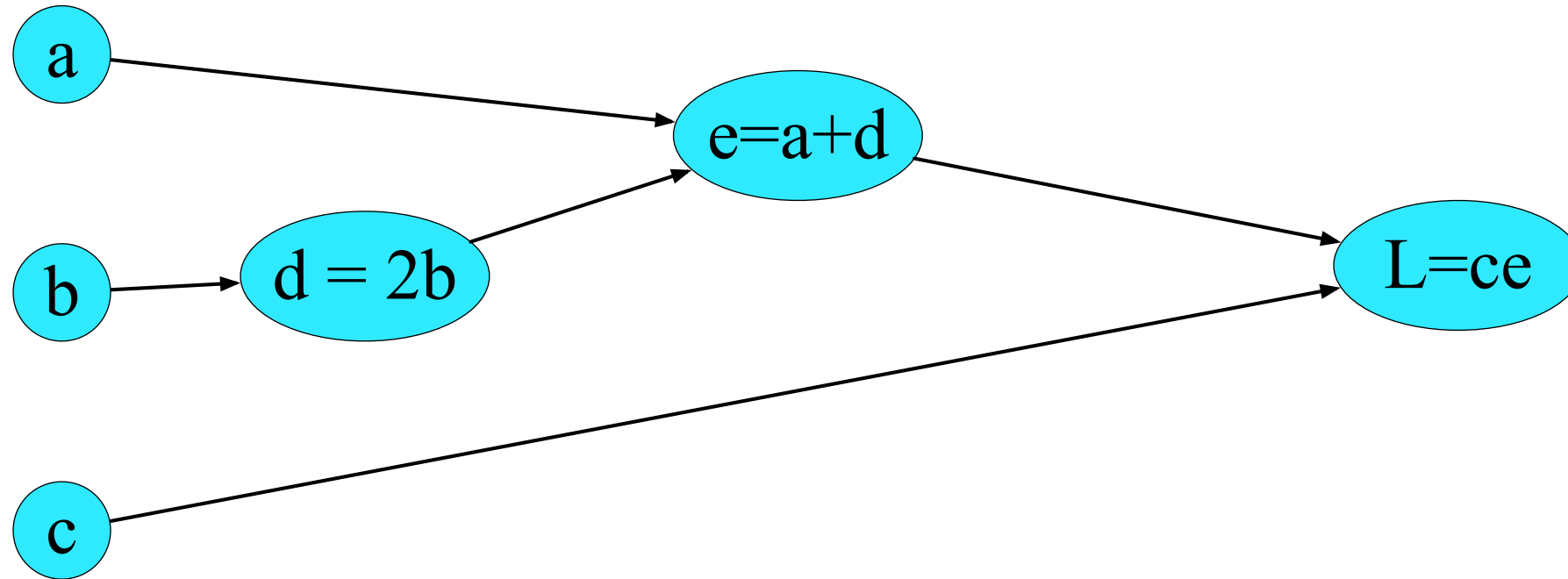
Example of a computational graph

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$



Backwards Differentiation

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

- **Backward pass** - we compute the derivatives that we'll need for the weight update
- It will help us obtain $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$ to update the parameters via gradient descent
- Recall: the derivative $\frac{\partial L}{\partial a}$ tells us how much a small change in a affects L

Recall: The Chain Rule

- Computing the derivative of a composite function:

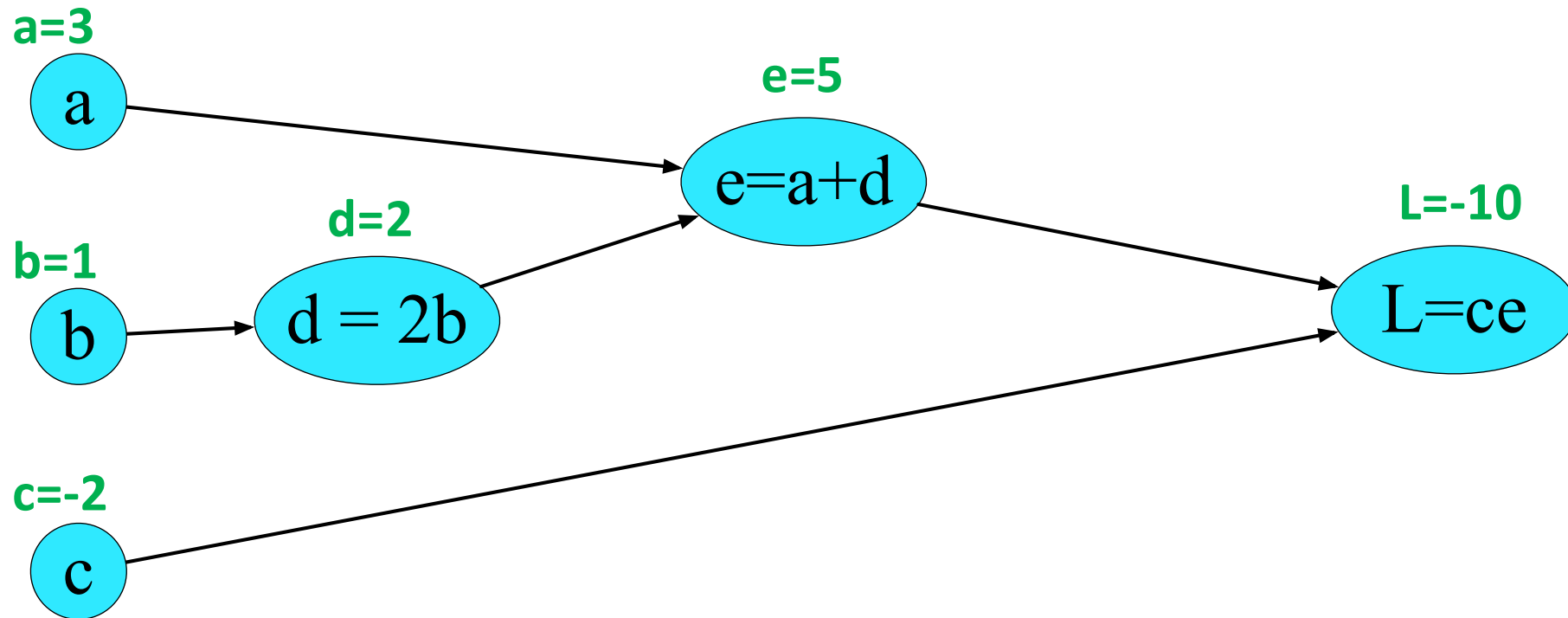
- $f(x) = u(v(x)) \quad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$

- $f(x) = u(v(w(x))) \quad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$

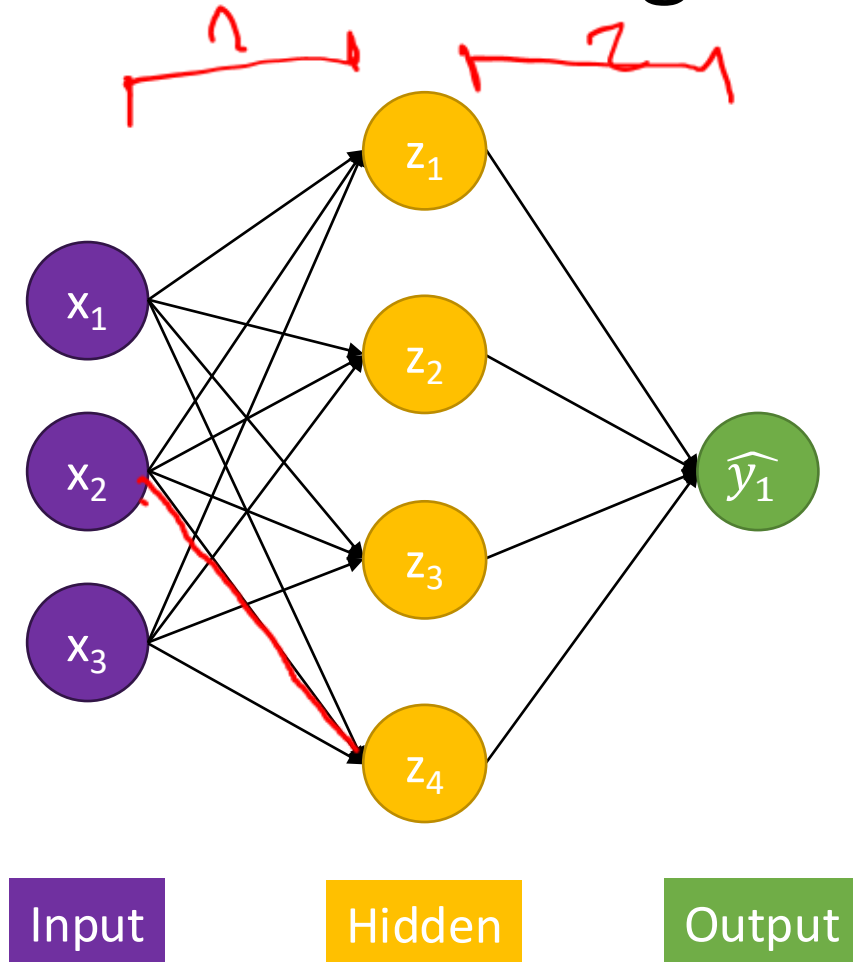
When poll is active respond at PollEv.com/ecbk



Calculate $\frac{\partial L}{\partial a}$ using the computational graph

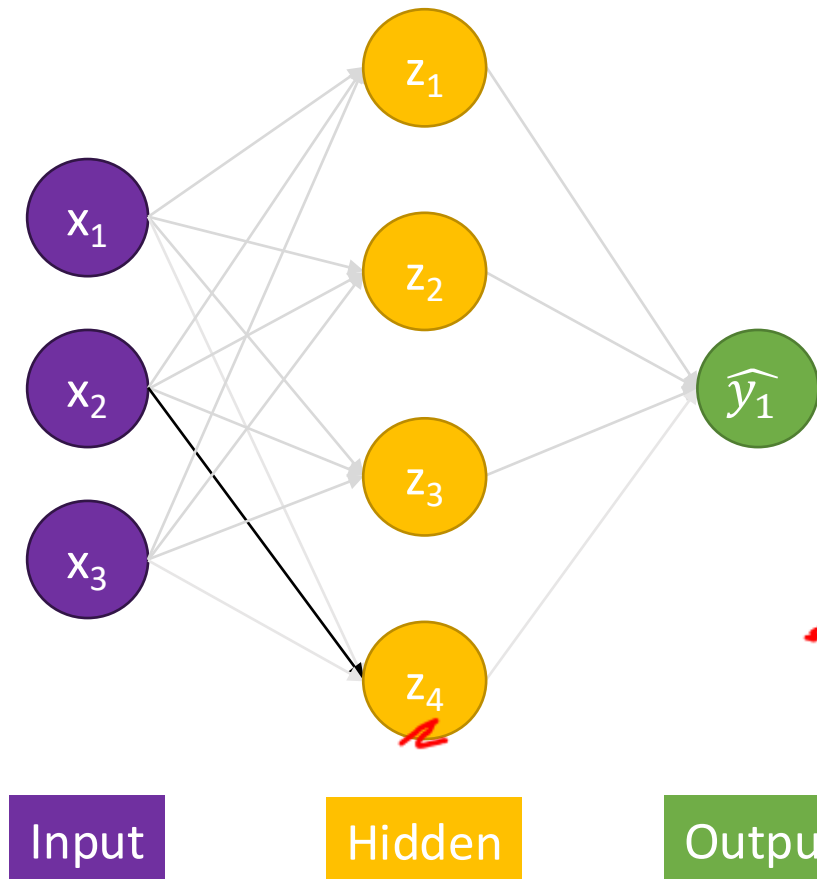


Back to training our 2-layer network



Let's step through it with focus on $w_{2,4}^{(1)}$

Back to training our 2-layer network



First, let's write down the forward pass equations containing $w_{2,4}^{(1)}$

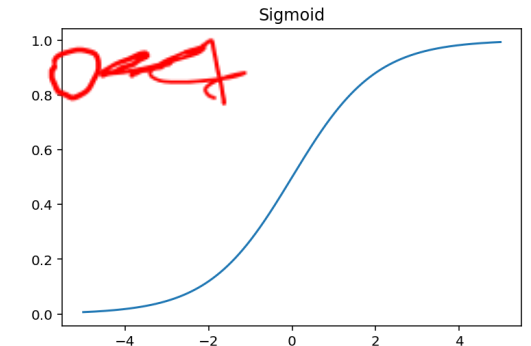
$$z_4 = g \left(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 w_{2,4}^{(1)} + x_3 w_{3,4}^{(1)} \right)$$

$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + z_4 w_{4,1}^{(2)}$$

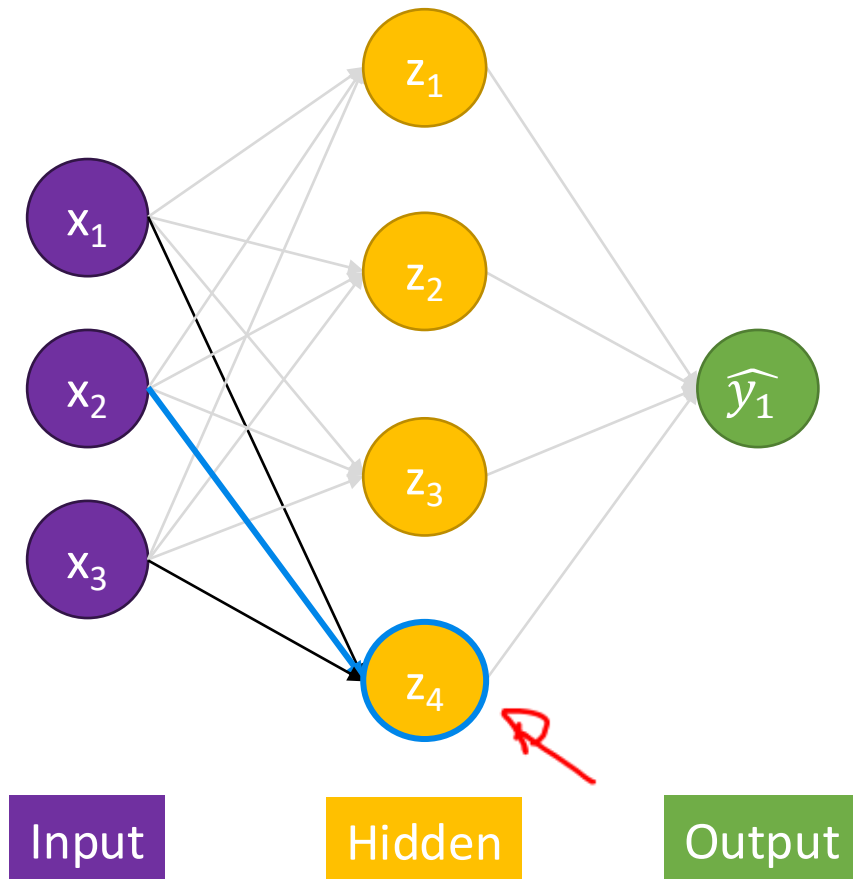
$\hat{y}_1 = \sigma(s_1)$
 $g \in [0, 1]$

$$\text{where } \sigma(s) = \frac{1}{1 + e^{-s}}$$

and g is ReLU



Back to training our 2-layer network

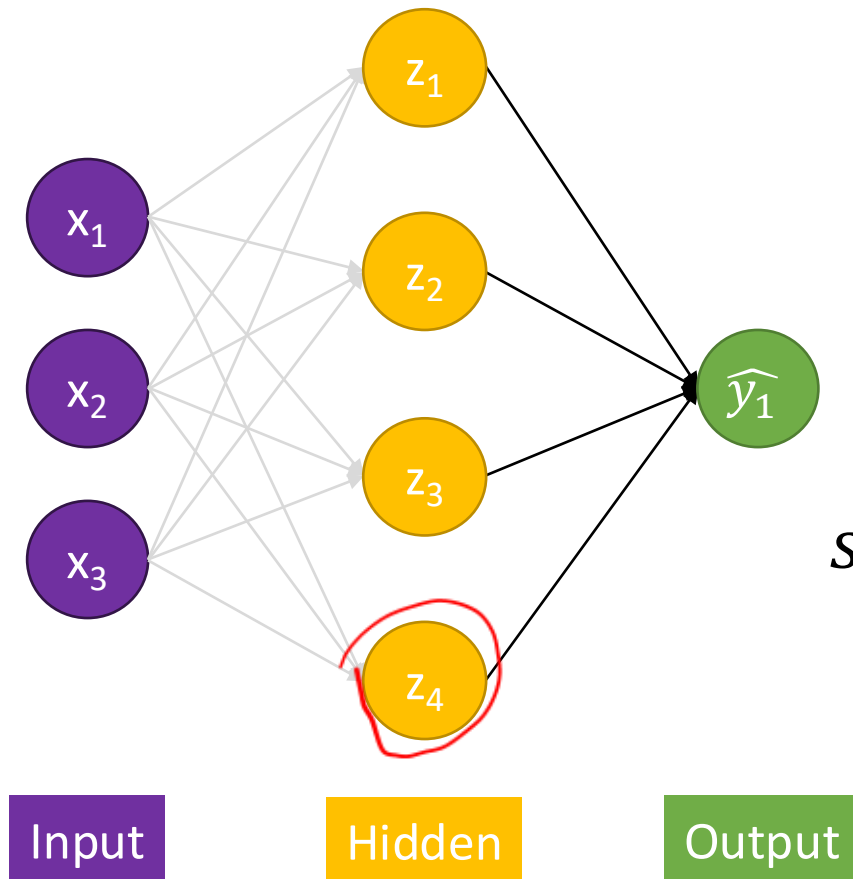


Let's step through it step by step with
focus on $w_{2,4}^{(1)}$

$$z_4 = g \left(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 \color{blue}{w_{2,4}^{(1)}} + x_3 w_{3,4}^{(1)} \right)$$

where g is ReLU

Back to training our 2-layer network



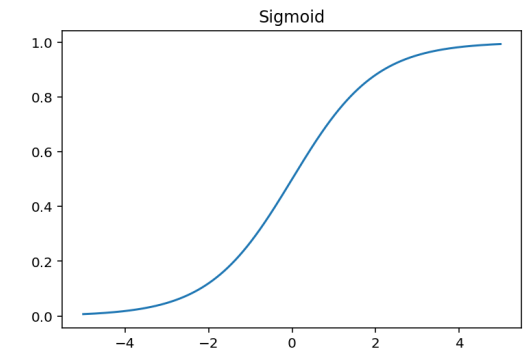
Let's step through it step by step
with focus on z_4

Now with the output layer:

$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + \textcolor{blue}{z_4} w_{4,1}^{(2)}$$

$$\hat{y}_1 = \sigma(\textcolor{blue}{s_1})$$

$$\text{where } \sigma(s) = \frac{1}{1+e^{-s}}$$



Recall: Binary Cross Entropy Loss

- A measure for how far off the current answer is to the right answer

$$Loss(y, \hat{y}) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- Derivative:

$$\frac{\partial Loss}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{(1 - y)}{(1 - \hat{y})}$$

Updating $w_{2,4}^{(1)}$

- We will need $\frac{\partial Loss}{\partial w_{2,4}^{(1)}}$

- Chain rule:

$$\frac{\partial Loss}{\partial w_{2,4}^{(1)}} = \underbrace{\frac{\partial Loss}{\partial \hat{y}_1}}_{\text{Derivative of the Loss}} \cdot \underbrace{\frac{\partial \hat{y}_1}{\partial s_1}}_{\text{Derivative of the activation (sigmoid)}} \cdot \underbrace{\frac{\partial s_1}{\partial z_4}}_{\text{Derivative of the weighted sum}} \cdot \underbrace{\frac{\partial z_4}{\partial w_{2,4}^{(1)}}}_{\text{Derivative of activation (ReLU) and weighted sum}}$$

- Then, update rule: $w_{2,4}^{(1)} \leftarrow w_{2,4}^{(1)} - \alpha \frac{\partial Loss}{\partial w_{2,4}^{(1)}}$

We will do the same for every weight in the neural network

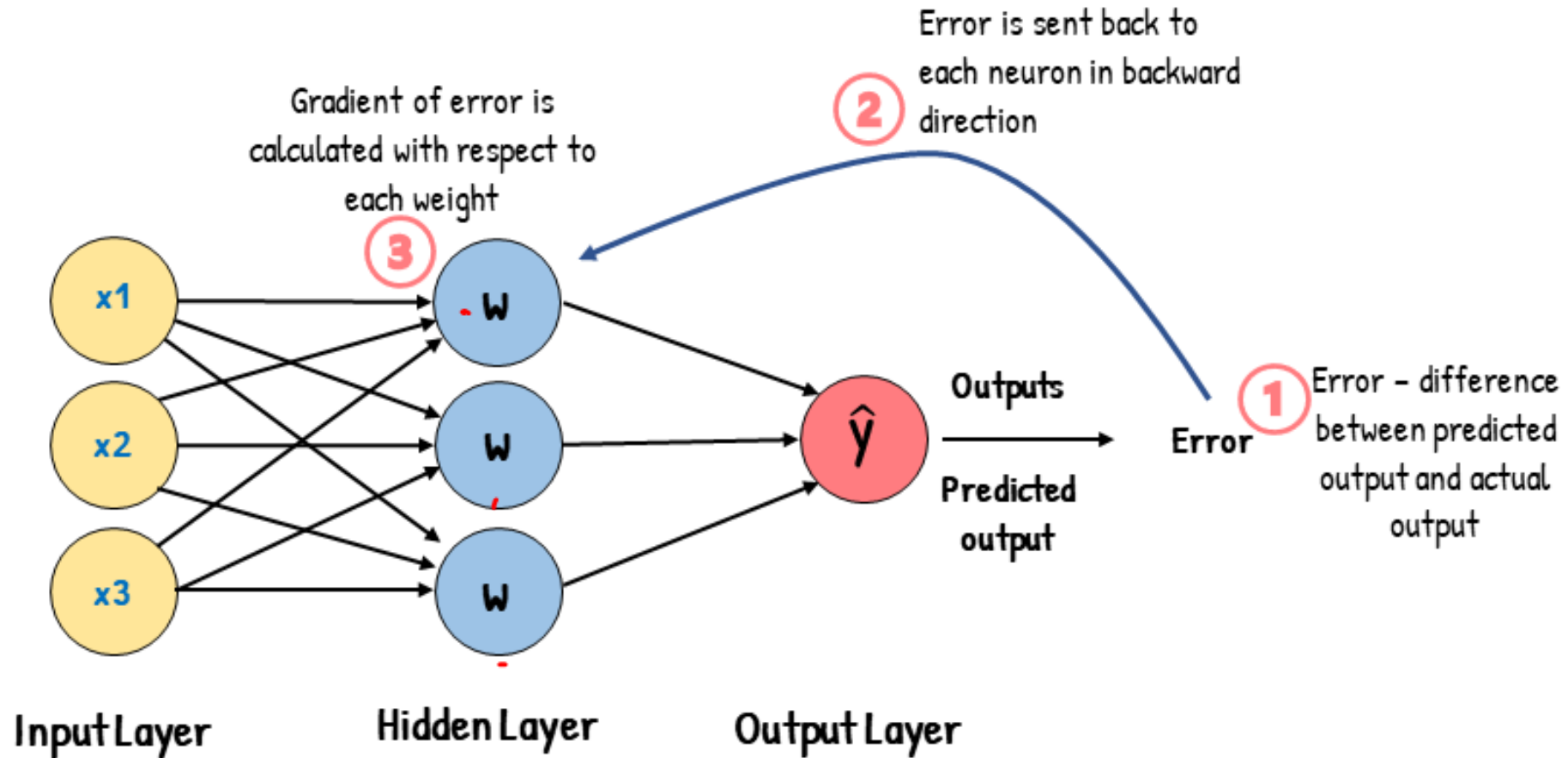
$$Loss = -[y \log \hat{y}_1 + (1 - y) \log(1 - \hat{y}_1)]$$

$$\hat{y}_1 = \sigma(s_1)$$

$$s_1 = w_{0,1}^{(2)} + z_1 w_{1,1}^{(2)} + z_2 w_{2,1}^{(2)} + z_3 w_{3,1}^{(2)} + z_4 w_{4,1}^{(2)}$$

$$z_4 = g(w_{0,4}^{(1)} + x_1 w_{1,4}^{(1)} + x_2 w_{2,4}^{(1)} + x_3 w_{3,4}^{(1)})$$

Summary: Backprop Computes Gradients needed for Gradient Descent Algorithm



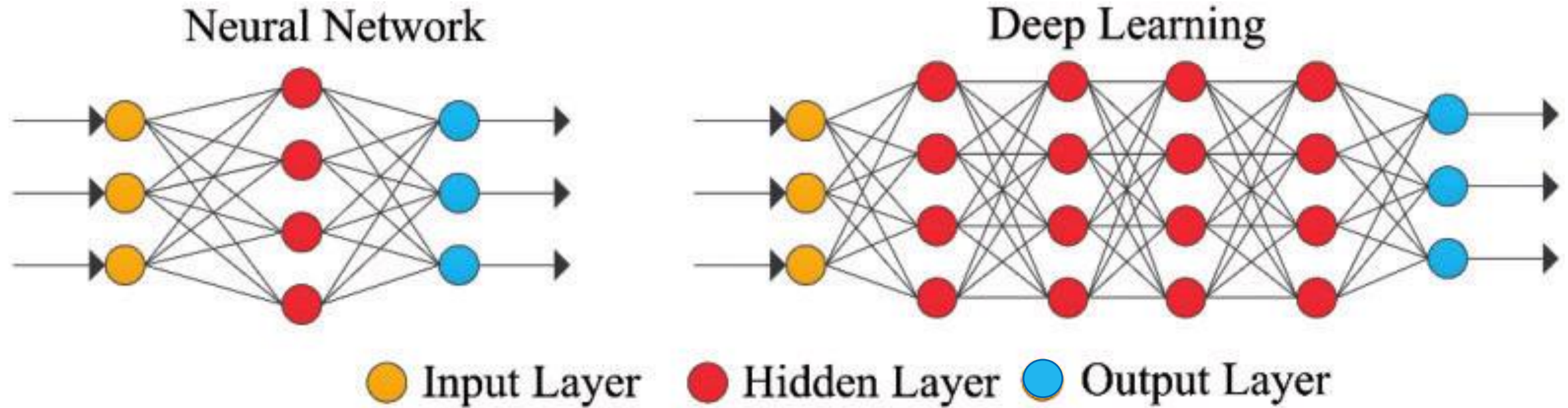
Source: <https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/>

Notes on Gradient Descent

- No convergence guarantees due to many local minima
- Problem of finding consistent weights is NP-complete
- Performance dependent on network structure
 - Need sufficient hidden nodes to express target
 - But too many nodes leads to overfitting + slow training!

Deep Learning: A Little Bit of History

From Neural Networks to Deep Learning

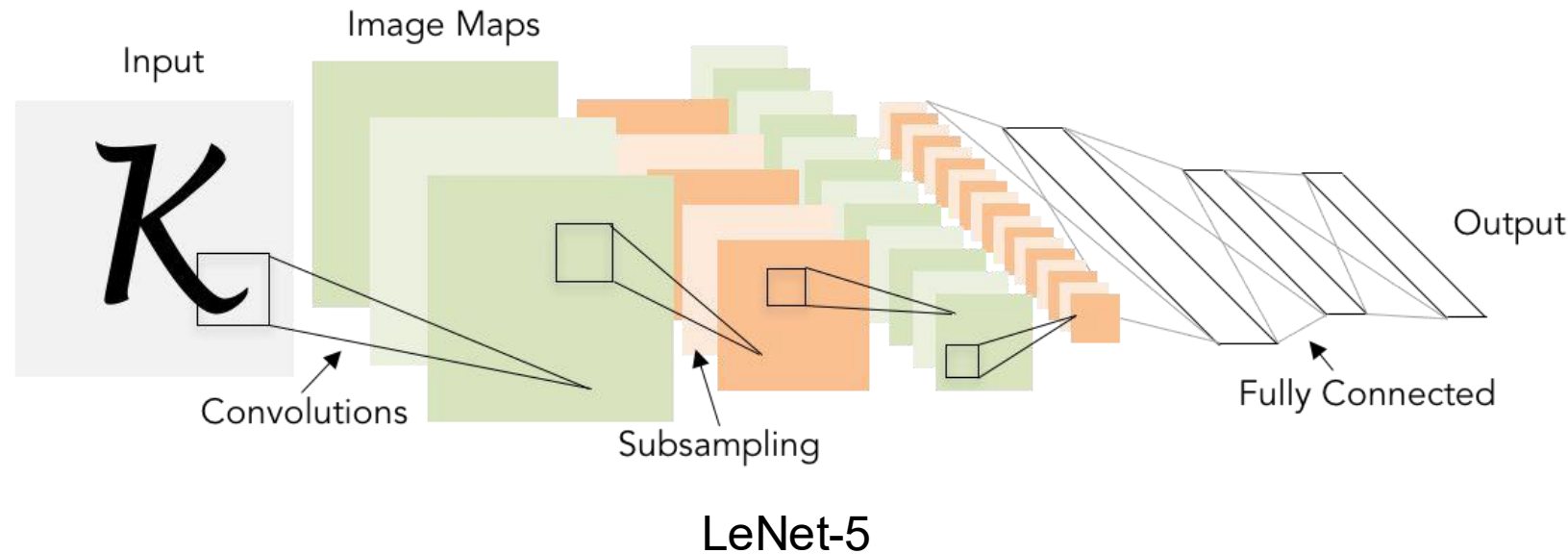


Source: Wanli Xing

A bit of history:

Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]



A bit of history:

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky, Sutskever, Hinton, 2012]

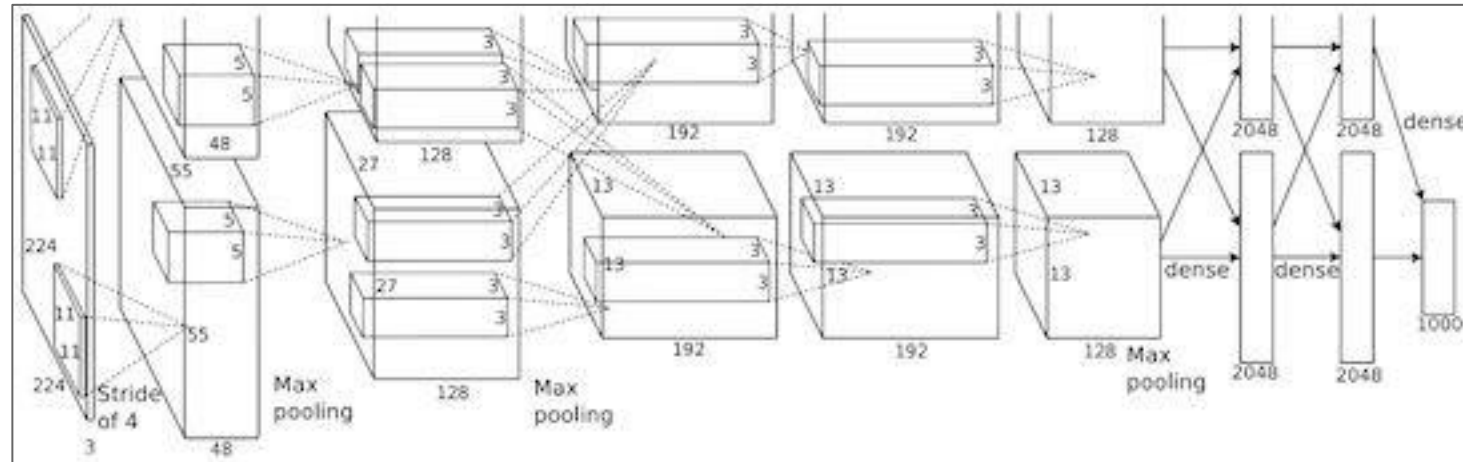
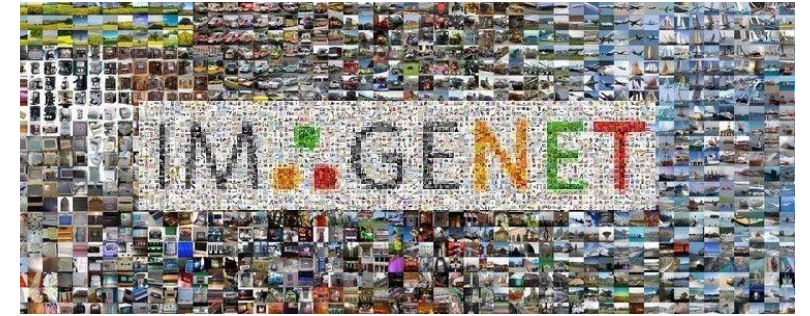


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

“AlexNet”

AlexNet Results

Classification



Retrieval



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

Convolutional Neural Nets (CNNs)

CNNs

Able to recognize spatial patterns in grid-like data.

Applications in computer vision

- Image classification
- Autonomous vehicles
- Facial recognition
- ...

But is sometimes applied to other tasks as well (time series data: sequence of vectors)

- Sentence classification
- Speech recognition
- ...

original: Fukushima, Kunihiko. "[Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position](#)." *Biological cybernetics* 36.4 (1980): 193-202.

breakthrough: Yann Lecun, Léon Bottou, Yoshua Bengio, Patrick Haffner. [Gradient-based learning applied to document recognition](#). Proceedings of the IEEE, 1998, 86 (11), pp.2278-2324. ff10.1109/5.726791ff. ffhal03926082f

Image Processing: Problems w/ FCNN

RGB image: three integers (0-255) per pixel

Red Green Blue

■ (0, 255, 0) is green

■ (144, 83, 255) is lighter purple

■ (60, 0, 168) is darker purple

1000x1000 RGB image:
1000x1000x3 = 3M integers

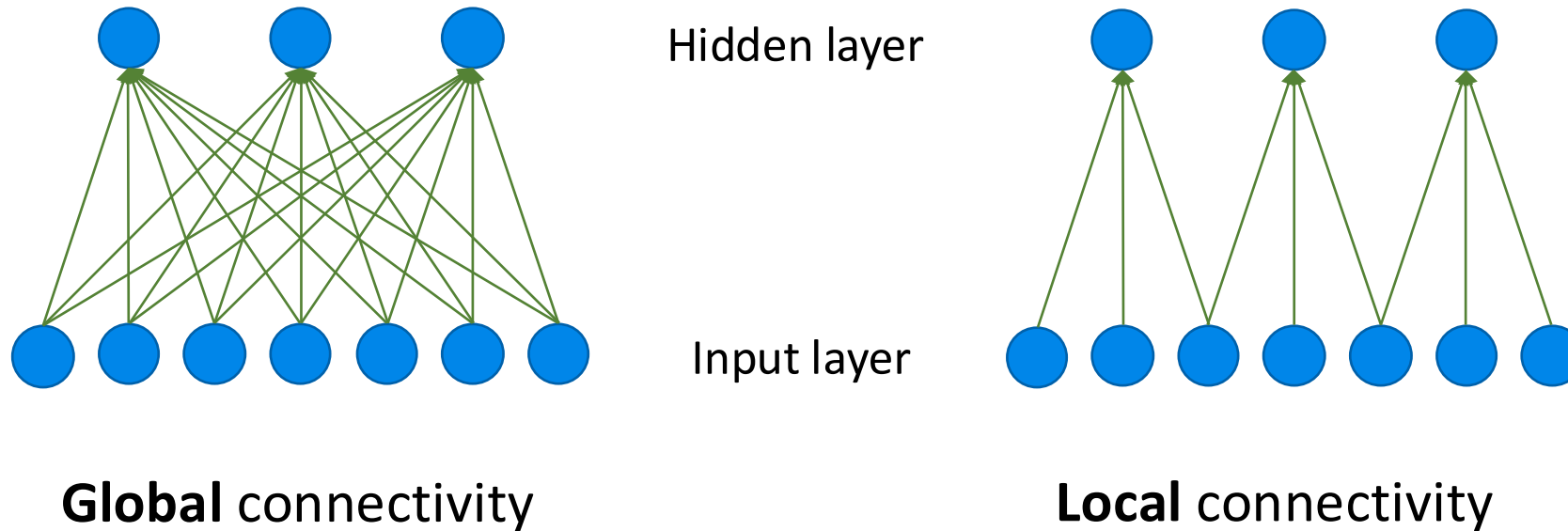
FCNN to input 1000x1000 RGB image, with 1000 hidden units
So $W^{[1]}$ would have shape (1000, 3M), so 3B weights!

- Great danger of overfitting
- Massive computational requirements
- The units in a FC layer don't know their spatial relationships.
- An image that's shifted a little bit appears completely different!

Design Principles of CNNs

 A Dog Spatial structure Pixels of an entity should be adequately distributed in space	Still a Dog  Spatial invariance The semantics of an entity should not be affected much by its position
--	--

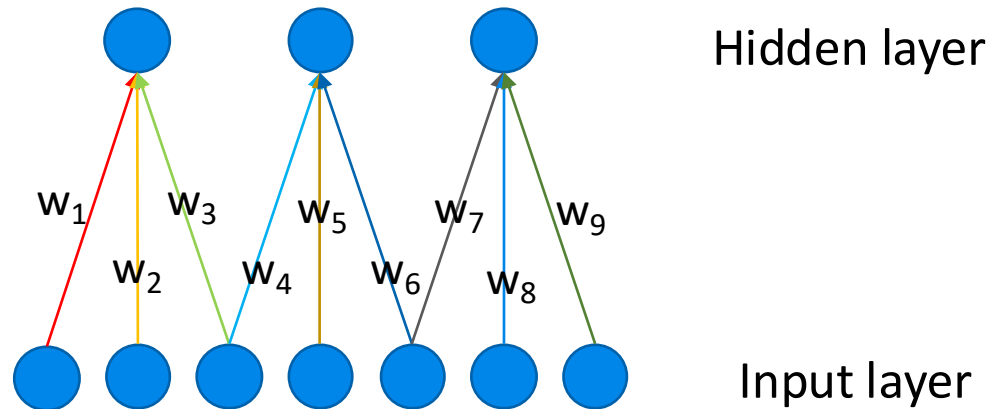
CNNs Property 1: Local Connectivity



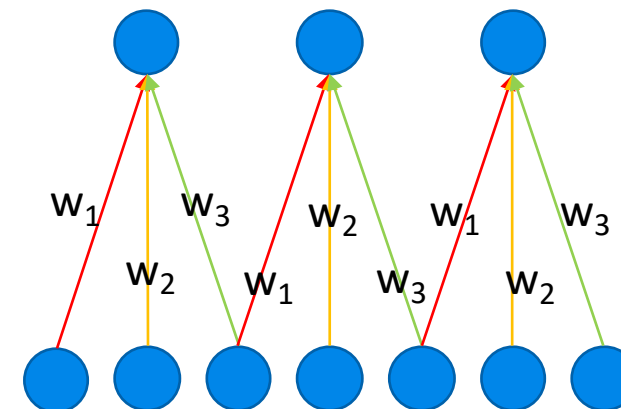
- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
 - Global connectivity: $3 \times 7 = 21$
 - Local connectivity: $3 \times 3 = 9$

Spatial structure helps us reduce the number of connections!

CNNs Property 2: Weight Sharing



Without weight sharing



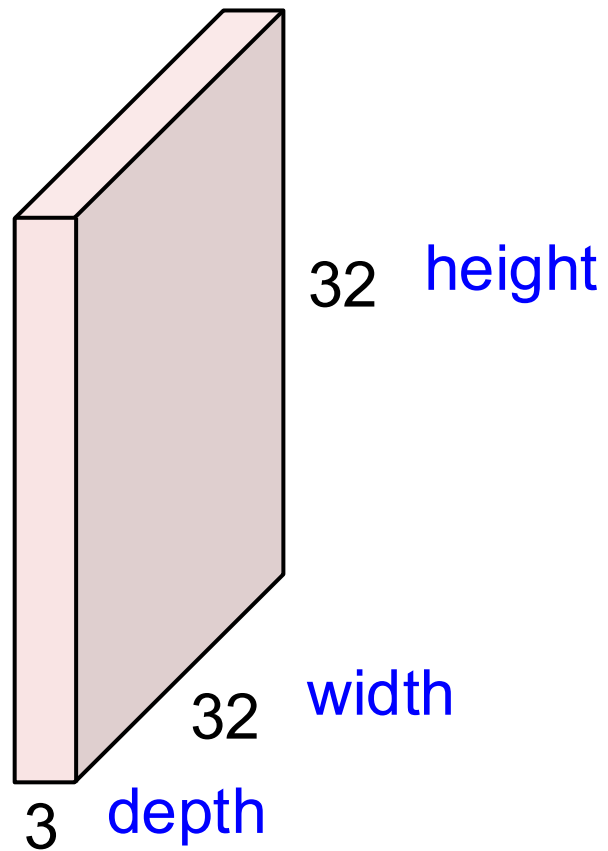
With weight sharing

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
 - Without weight sharing: $3 \times 3 = 9$
 - With weight sharing : $3 \times 1 = 3$

Spatial invariance helps us reduce number of weights (parameters to learn)!

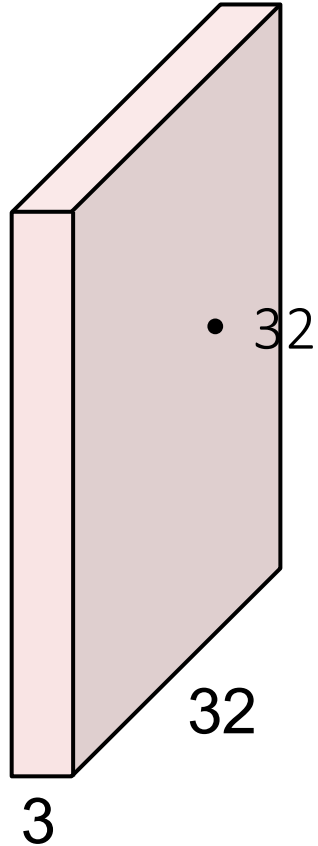
Convolution Layer

32x32x3 image -> preserve spatial structure

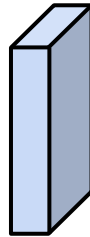


Convolution Layer

- 32x32x3 image



- 5x5x3 filter

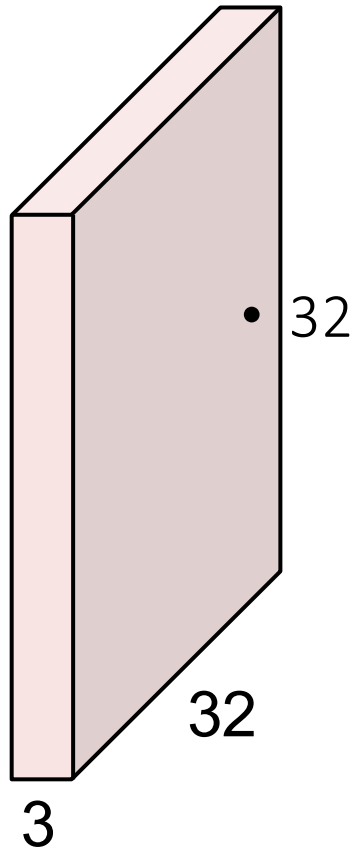


- **Convolve** the filter with the image
- i.e. “slide over the image spatially, computing dot products”

Convolution Layer

Filters always extend the full depth of the input volume

- 32x32x3 image



- 5x5x3 filter



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

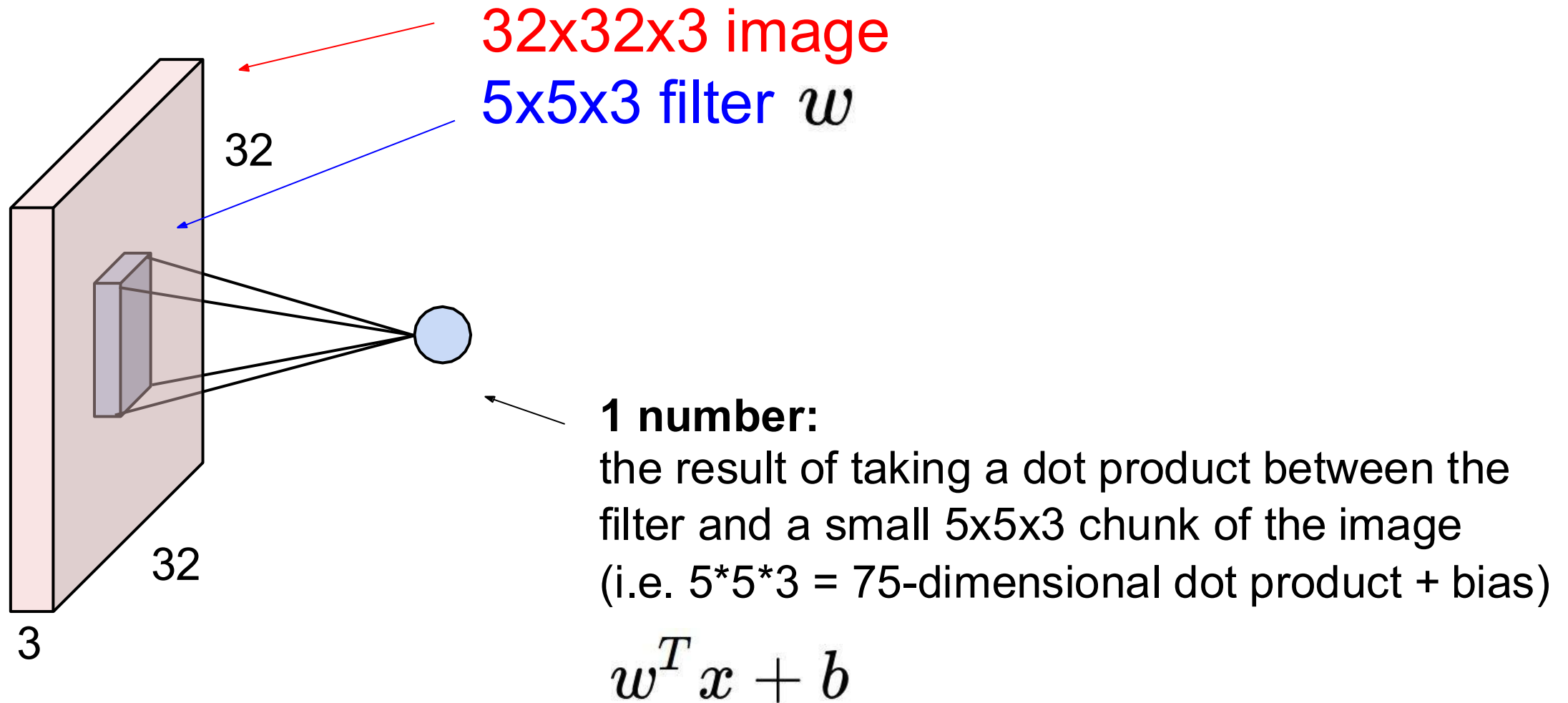
Image

4		

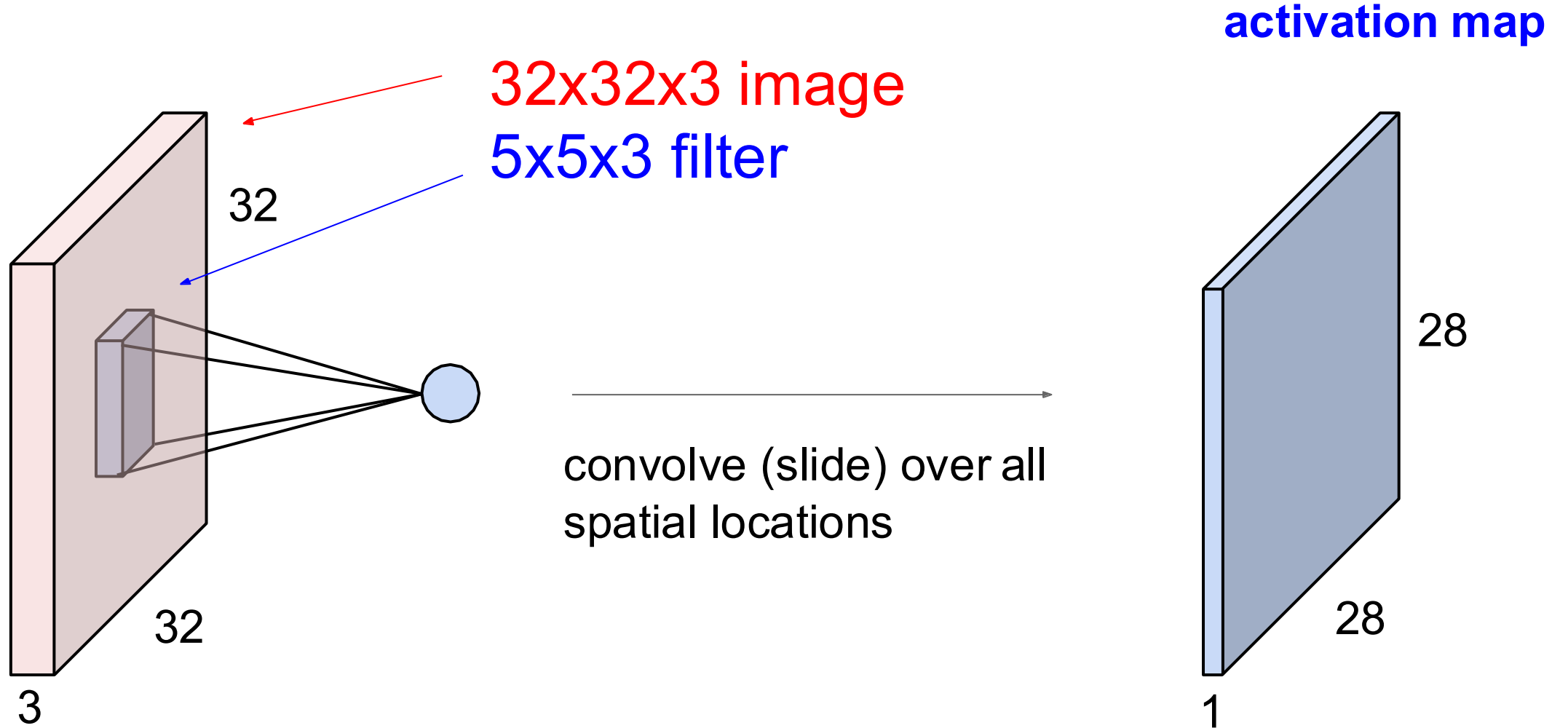
Convolved
Feature

- **Convolve** the filter with the image
- i.e. “slide over the image spatially, computing dot products”

Convolution Layer



Convolution Layer

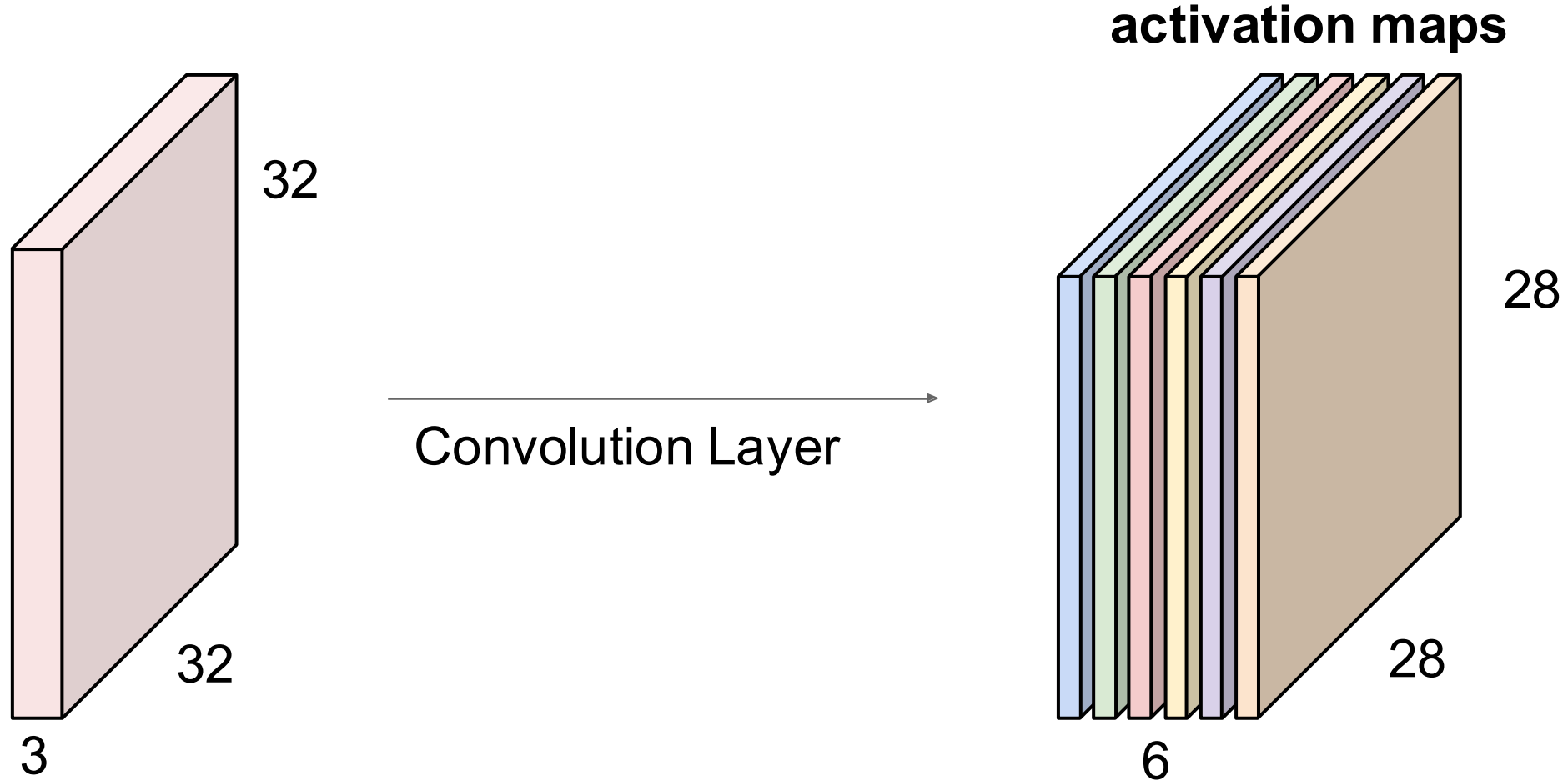


Convolution Layer

consider a second, **green** filter

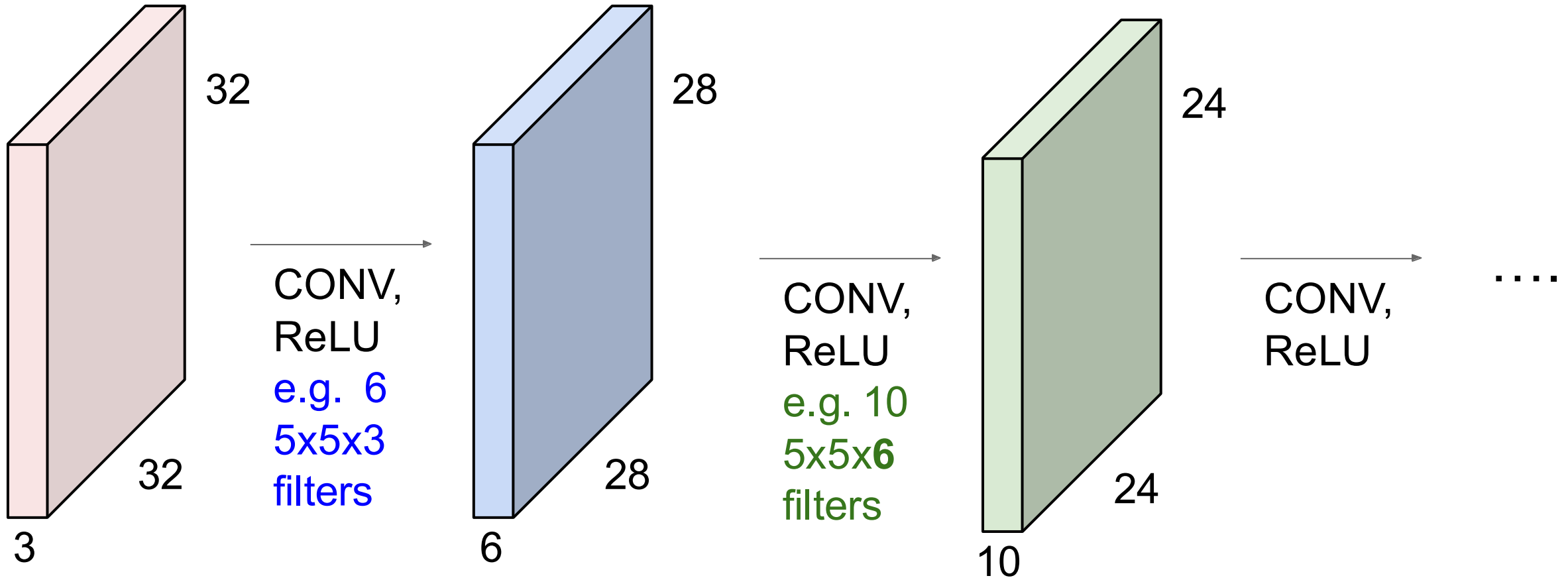


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



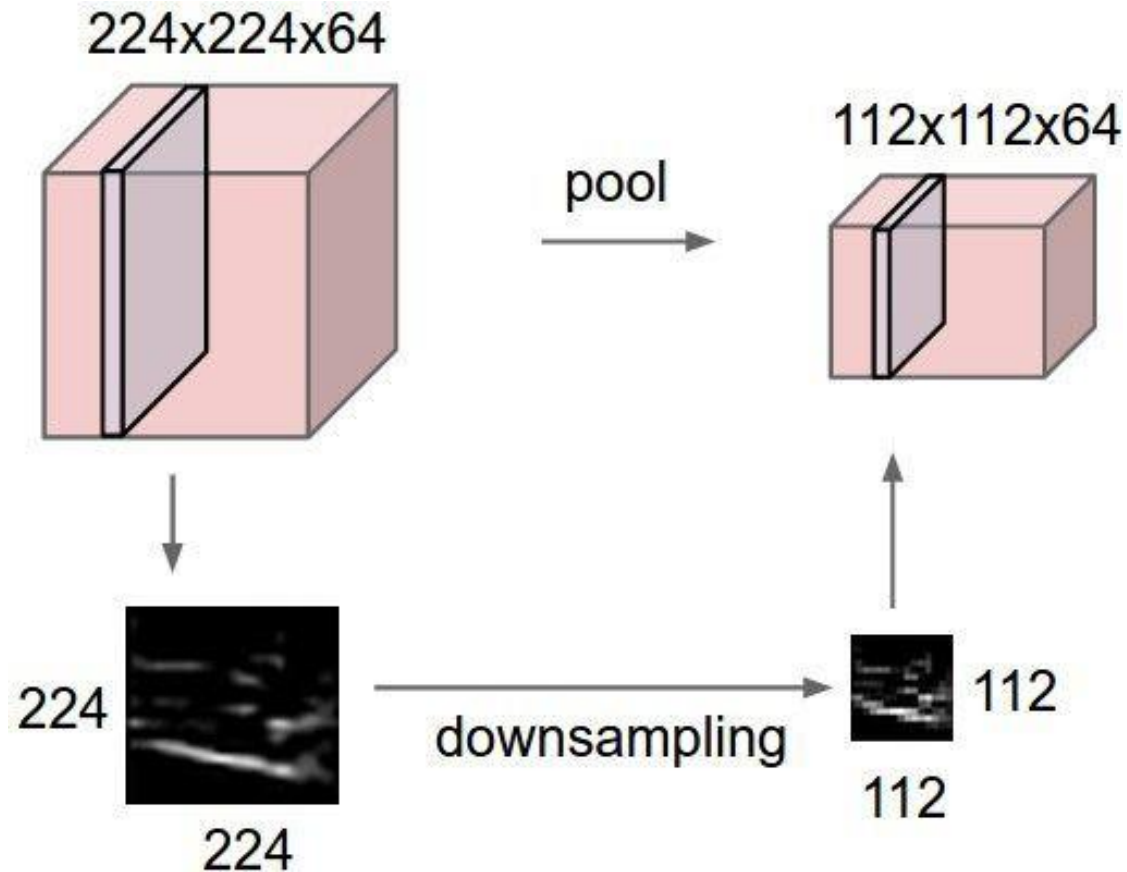
We stack these up to get a “new image” of size 28x28x6!

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

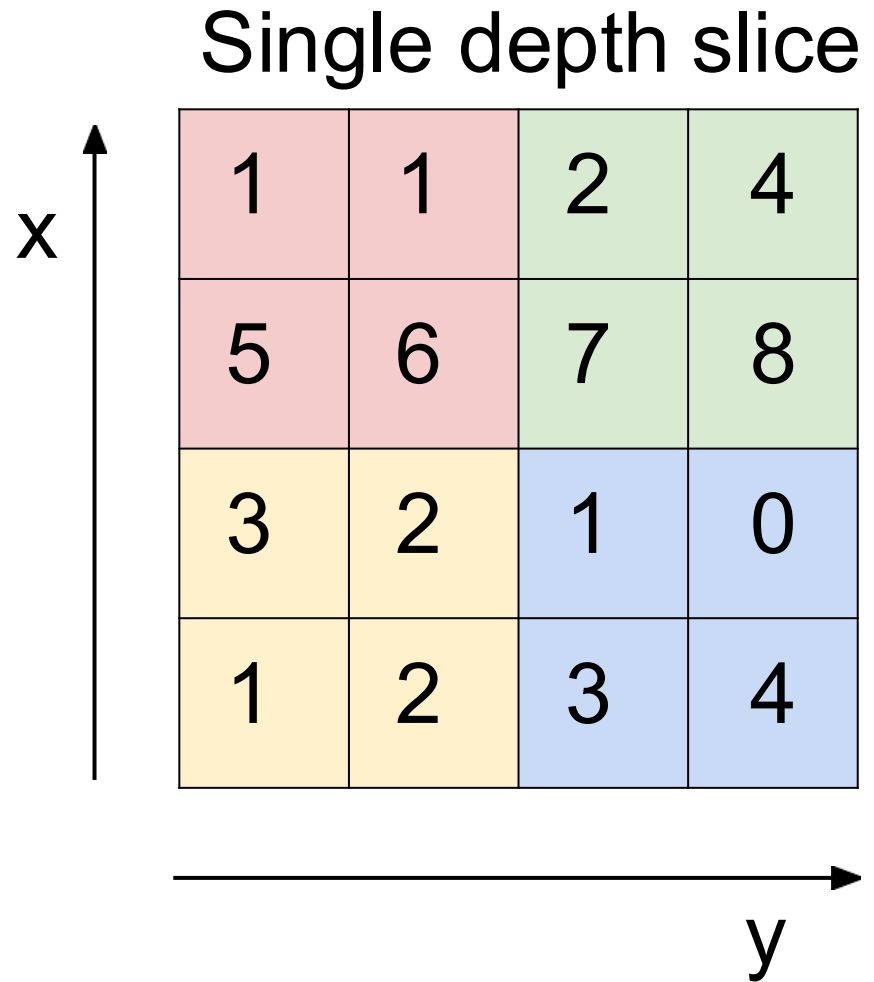


Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:



MAX POOLING



max pool with 2x2 filters
and stride 2 ()

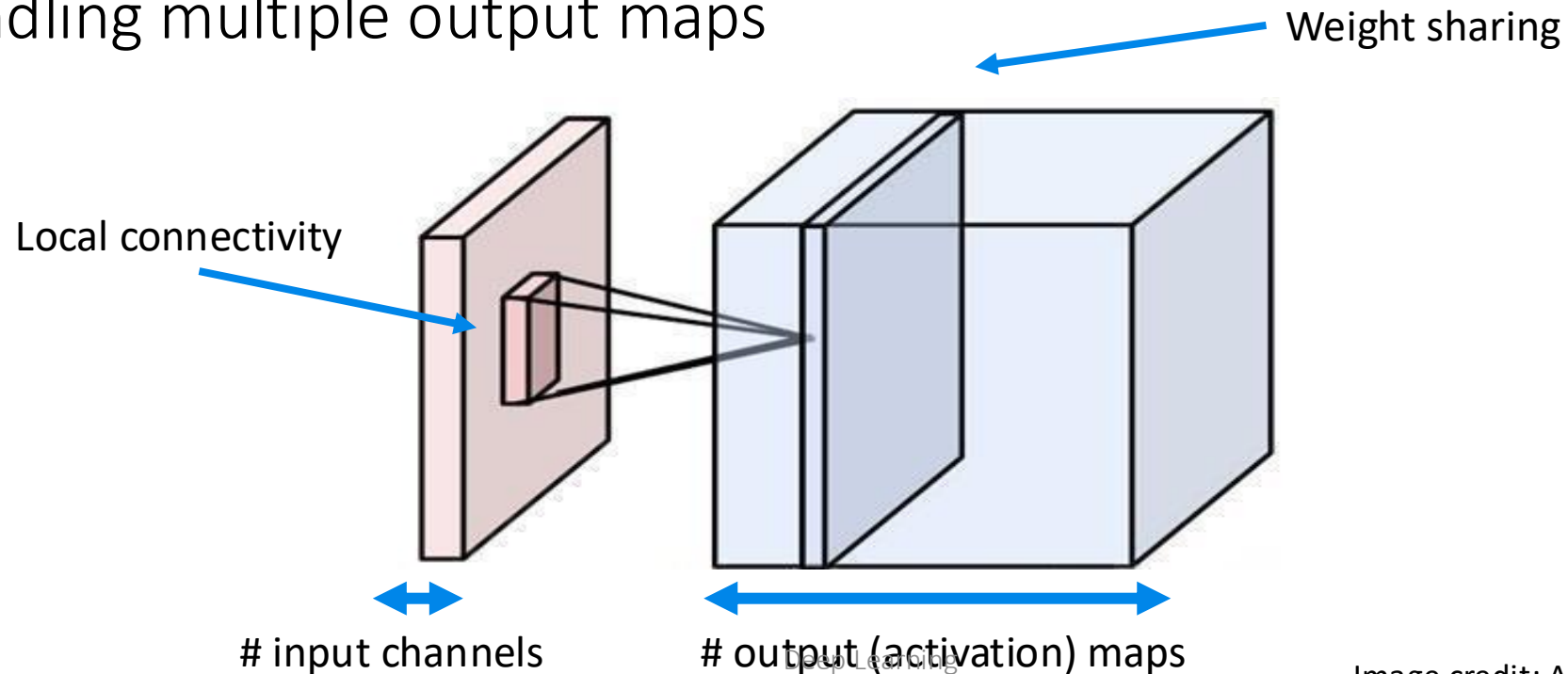


6	8
3	4

Stride: number of pixels we're
stepping over as we slide our filter
across the input image

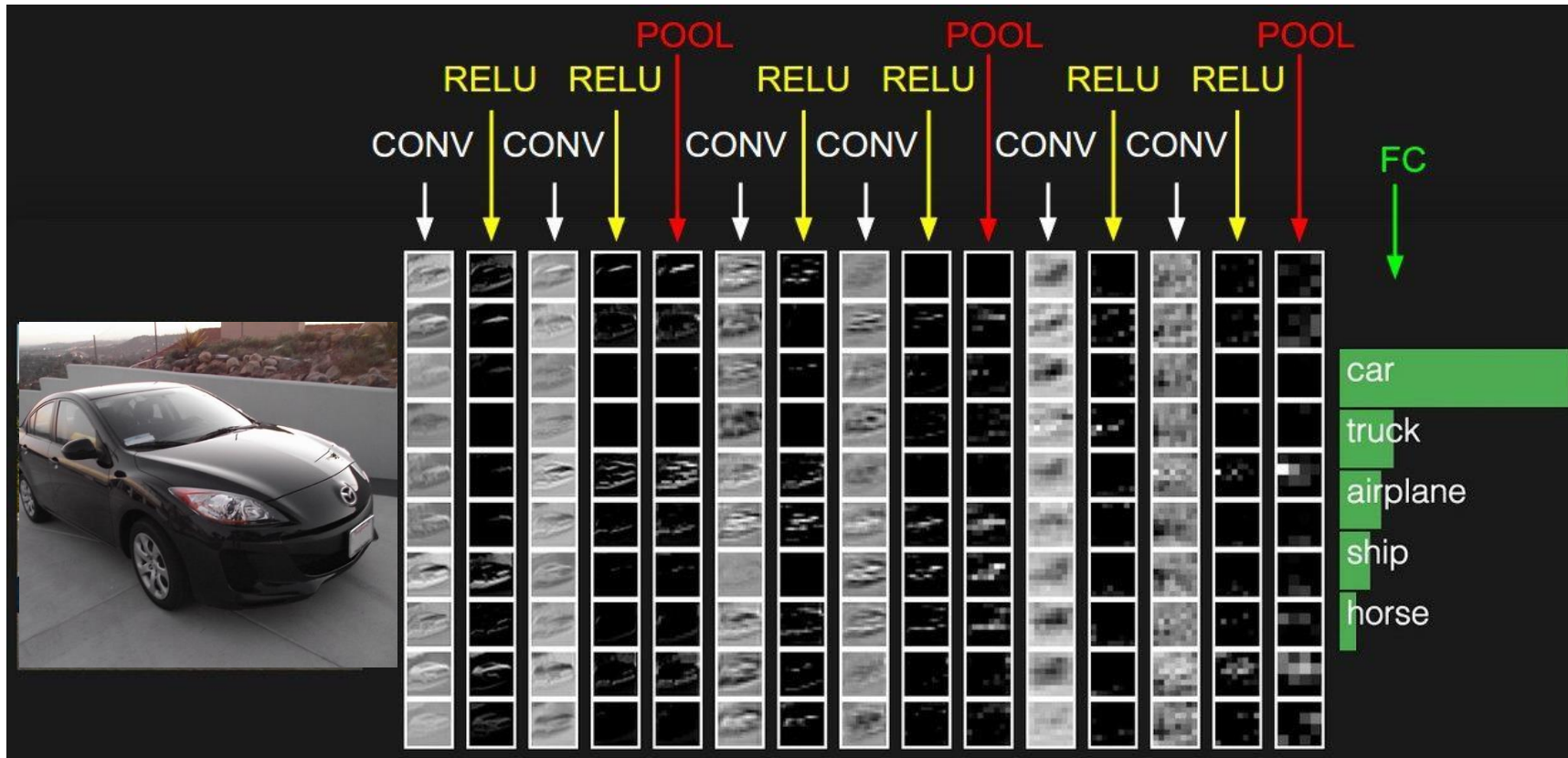
Putting them together

- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps



Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



Case Study: AlexNet

[Krizhevsky et al. 2012]

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

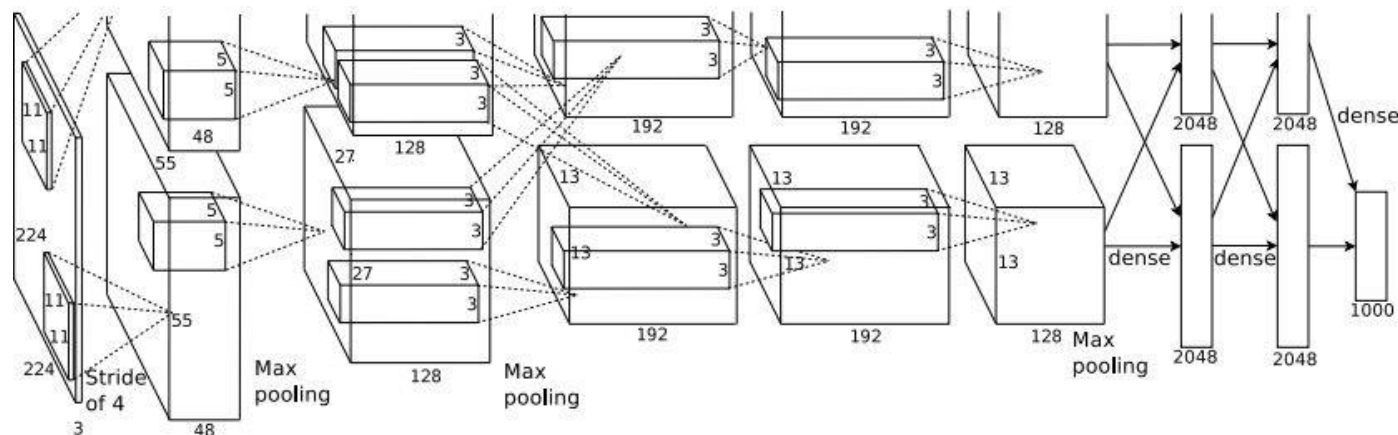
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



- First deep learning model to utilize Graphics Processing Units (GPUs) for training
- Deeper than previous neural nets
- + several other tricks, e.g., **dropout**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.